

Python ile Sayısal İşaret İşleme

Modülasyon, Demodülasyon ve IQ İşleme

NumPy • SciPy • Matplotlib ile Sıfırdan Tam Bir SDR Alıcısı

Bu belge önce **örnekleme**, **Nyquist ve karmaşık sayılar** gibi temelleri; sonra **DFT/FFT ile frekans analizini**; ardından **FIR/IIR filtreleri**; sonra işin kalbi olan **IQ (karmaşık bant temeli)** kavramını; en sonda ise **AM, FM, PM ve sayısal modülasyonların** tam modem örneklerini anlatır. Bütün kodlar **NumPy/SciPy** ile vektörleştirilmiş, çalıştırılabilir Python'dur.

Kapsam — Kısım 1 (Temeller): Python DSP ekosistemi • işaret nedir • örnekleme & Nyquist • aliasing • kuantalama & ADC • karmaşık sayılar & fazörler • sinyal üretimi & WAV G/Ç.

Kapsam — Kısım 2 (Frekans Analizi): Fourier fikri • DFT • `numpy.fft` • frekans eksenini & ölçekleme • radix-2 FFT'yi kendin yaz • pencereleme • periyodogram, Welch, spektrogram.

Kapsam — Kısım 3 (Filtreler): konvolüsyon • FIR & doğrusal faz • `firwin/remez` ile tasarım • IIR, `sos` ve `filtfilt` • kutup-sıfır • hareketli ortalama, CIC, notch • blok işleme.

Kapsam — Kısım 4 (IQ & Bant Temeli): analitik işaret & Hilbert • I/Q anlamı • NCO & mikser • desimasyon / interpolasyon • polifaz • DC ofset & IQ dengesizliği.

Kapsam — Kısım 5 (Modülasyon): AM (zarf & senkron) • FM (IQ diskriminatör, Carson, Bessel) • PM • ASK/FSK/PSK/QAM • darbe şekillendirme & göz diyagramı • PLL, Costas, zamanlama kurtarma.

Kapsam — Kısım 6 (Uçtan Uca Alıcı): IQ dosya formatları • RTL-SDR akışı • tam WBFM radyo hattı • de-emphasis • performans & gerçek zamanlı akış.

Kapsam — Kısım 7 (İleri): stereo FM çözücü • AGC • uçtan uca QPSK modem & BER • OFDM • konvolüsyonel kod & Viterbi.

Kapsam — Kısım 8 (Projeler): DTMF çözücü (Goertzel) • ses üzerinden FSK modem • gerçek zamanlı spektrum analizörü • enstrüman akort cihazı.

Kapsam — Kısım 9 (Matematik): diklik & Euler • Fourier zinciri (FS/FT/DTFT/DFT) • FFT ispatı • örnekleme teoreminin türetimi • konvolüsyon & modülasyon teoremleri • Parseval • anlık frekans.

Kapsam — Kısım 10 (Referans): NumPy/SciPy sözlüğü • hızlı tarif kitabı • sık yapılan hatalar.

Hazırlanma Tarihi: 9 Eylül 2026

Python 3 • NumPy • SciPy • Nyquist-Shannon • RTL-SDR

Contents

1	Python DSP Araç Kutusu	4
1.1	Kurulum	4
1.2	Hangi İş İçin Hangi Kütüphane?	4
1.3	Rehber Boyunca Kullanılan Ortak Başlık	5
2	İşaret Nedir? Analogdan Sayısala	5
2.1	Ayrık-Zamanlı İşaretin NumPy'deki Karşılığı	6
2.2	Veri Tipleri: Bellek ve Hız	6
3	Örnekleme, Nyquist ve Aliasing	7
3.1	Aliasing: Kimlik Değiştiren Frekanslar	7
3.2	Aliasing'i Duymak	8
4	Kuantalama ve ADC	9
4.1	Dither: Gürültü Ekleyerek Kaliteyi Artırmak	9
5	Karmaşık Sayılar ve Fazörler	10
5.1	Neden Negatif Frekans Var?	10
6	Sinyal Üretimi ve WAV Dosyaları	11
6.1	Temel Dalga Biçimleri	11
6.2	WAV Okuma ve Yazma	11
6.3	Gerçek Zamanlı Ses Girişi/Çıkışı	12
7	Fourier Fikri	13
7.1	DFT'yi Doğrudan Yazmak	13
7.2	Frekans Eksen ve Ölçekleme	14
7.3	Çözünürlük, Sızıntı ve Sıfır Doldurma	14
8	FFT: Hızlı Fourier Dönüşümü	15
8.1	Yerinde (In-Place) Yinelemeli FFT	15
8.2	Hız Karşılaştırması	16
9	Pencereleme (Windowing)	17
10	Güç Spektral Yoğunluğu ve Spektrogram	18
10.1	Spektrogram: Zaman-Frekans Görüntüsü	18
10.2	IQ İşaretlerin Spektrumu	19
11	Konvolüsyon ve FIR Filtreler	20
11.1	NumPy/SciPy ile Konvolüsyon	20
11.2	FIR Filtrenin Doğrusal Fazı	21
11.3	Pencere Yöntemiyle Alçak Geçiren Tasarım	21
11.4	Filtreyi Doğrulamak: Frekans Yanıtı	21
11.5	Tasarım Parametreleri: Kaç Katsayı Gerekir?	22
12	IIR Filtreler, SOS ve Sıfır Faz	22
12.1	Sıfır Faz: filtfilt	23
12.2	Biquad: IIR'ın Yapı Taşı	23

13 Ucuz Filtreler: Hareketli Ortalama, CIC, DC Engelleyici	24
13.1 Hareketli Ortalama	24
13.2 CIC (Cascaded Integrator-Comb)	24
13.3 DC Engelleyici	24
14 Akış İşleme: Blok Blok Filtreleme	25
15 IQ Nedir? Analitik İşaret	26
15.1 Analitik İşaret ve Hilbert Dönüşümü	26
15.2 IQ'nun Üç Yorumu	27
16 NCO, Mikser ve Frekans Kaydırma	27
16.1 Gerçek İşareti Bant Temeline İndirmek	28
17 Desimasyon, İnterpolasyon ve Polifaz	28
17.1 Polifaz Neden Hızlı?	29
17.2 IQ Desimasyonu	29
18 IQ Kusurları: DC Ofset, Dengesizlik ve Düzeltme	29
18.1 Takımyıldız (Constellation) Çizimi	30
19 Genel Bakış: Modülasyon Neden Var?	31
20 AM: Genlik Modülasyonu	31
20.1 AM Modülatörü	32
20.2 Zarf Detektörü ile Demodülasyon	32
20.3 Senkron (Coherent) Demodülasyon	33
21 FM: Frekans Modülasyonu	33
21.1 Bessel: FM Spektrumu Neden Geniş?	33
21.2 FM Modülatörü	34
21.3 FM Demodülasyonu: IQ Diskriminatör	34
21.4 De-emphasis: FM'in Son Rötuşu	35
22 PM: Faz Modülasyonu	35
23 Sayısal Modülasyonlar: ASK, FSK, PSK, QAM	36
23.1 BPSK ve QPSK	36
23.2 FSK	36
23.3 QAM	37
24 Darbe Şekillendirme ve Göz Diyagramı	37
24.1 Göz Diyagramı	38
25 Taşıyıcı ve Zamanlama Kurtarma	39
25.1 Faz Kilitlemeli Döngü (PLL)	39
25.2 Costas Döngüsü: PSK için Taşıyıcı Kurtarma	39
25.3 Sembol Zamanlama Kurtarma: Gardner	40
26 IQ Dosya Formatları ve RTL-SDR	41
26.1 Donanımdan Canlı Akış	41
27 Tam WBFM Alıcı Hattı	42
27.1 Dar Bant FM (NFM) Farkı	44

28 Performans ve Gerçek Zamanlı Akış	44
28.1 Ne Kadar Hızlı Olmalı?	44
28.2 Üretici–Tüketici ile Gerçek Zamanlı Alıcı	44
28.3 Nereye Bakmalı?	45
29 Stereo FM Çözücü	46
30 Otomatik Kazanç Kontrolü (AGC)	47
31 Uçtan Uca Sayısal Modem: QPSK	48
31.1 Verici	48
31.2 Kanal Modeli	48
31.3 Alıcı	49
31.4 BER Ölçümü	49
32 OFDM: Çok Taşıyıcılı Modülasyon	50
32.1 Döngüsel Önek (Cyclic Prefix) Neden Var?	51
33 Kanal Kodlaması: Konvolüsyonel Kod ve Viterbi	51
34 Proje 1 — DTMF (Telefon Tuşu) Çözücü	53
35 Proje 2 — Ses Üzerinden Veri İletimi (FSK Modem)	54
36 Proje 3 — Gerçek Zamanlı Spektrum Analizörü	55
37 Proje 4 — Enstrüman Akort Cihazı	56
38 Karmaşık Üsteller ve Diklik	58
39 Fourier Zinciri: Seriden DFT’ye	58
39.1 DTFT’den DFT’ye: Kodun Türetimi	59
39.2 FFT’nin Böl-ve-Fethet İspatı	59
40 Örnekleme Teoreminin Türetimi	59
41 Konvolüsyon, Modülasyon ve Parseval Teoremleri	60
42 Analitik İşaret ve Anlık Frekans	61
42.1 Polar Diskriminatörün Türetimi	61
42.2 Hilbert Dönüşümü ile Analitik İşaret	61
43 Fonksiyon Sözlüğü	63
43.1 Spektral Analiz	63
43.2 Filtre Tasarımı ve Uygulama	63
43.3 Hız Dönüşümü ve Modülasyon	64
44 Hızlı Tarif Kitabı	64
45 Sık Yapılan Hatalar	65
46 Genel Bakış: Zincir Nasıl Bağlanıyor?	66

KISIM 1

İşaret İşlemenin Temelleri

1. Python DSP Araç Kutusu

Sayısal işaret işleme, uzun sayı dizileri üzerinde milyonlarca çarpma-toplama demektir. Python'un kendisi bu iş için yavaştır; ama **NumPy** ve **SciPy**, altta C/Fortran (BLAS, FFTPACK/pocketfft) çağıran *vektörleştirilmiş* işlemler sunar. Doğru yazılmış NumPy kodu, elle yazılmış C koduna yakın hızda çalışır — üstelik onlarca kat daha kısadır.

1.1 Kurulum

```
# Sanal ortam
$ python3 -m venv dsp && source dsp/bin/activate

# Çekirdek paketler
$ pip install numpy scipy matplotlib

# Ses giriş/çıkışı ve dosya
$ pip install soundfile sounddevice

# SDR donanımı (isteğe bağlı)
$ pip install pyrtlsdr

# Interaktif çalışma
$ pip install jupyter
```

1.2 Hangi İş İçin Hangi Kütüphane?

Paket	DSP'deki rolü
numpy	Dizi aritmetiği, karmaşık sayılar, numpy.fft ile FFT. Her şeyin temeli.
scipy.signal	Filtre tasarımı, konvolüsyon, yeniden örnekleme, pencereler, spektral analiz.
scipy.fft	numpy.fft'ten hızlı; çok çekirdekli (workers), gerçek FFT ve DCT.
matplotlib	Dalga biçimi, spektrum, spektrogram, takımyıldız, göz diyagramı.
soundfile	WAV/FLAC okuma-yazma (libsndfile sarmalayıcısı).
sounddevice	Gerçek zamanlı ses giriş/çıkışı (PortAudio).
pyrtlsdr	RTL-SDR donanımından IQ örnek akışı.
numba / pyfftw	Darboğaz döngüleri JIT ile hızlandırma, FFTW bağlaması.

Dikkat

DSP'de **Python döngüsü yazmak** en sık yapılan hatadır. `for n in range(len(x)): y[n] = ...` biçimindeki bir kod, aynı işin NumPy karşılığından 50–500 kat yavaştır. Kural: *örnek başına düşünmeyi bırak, dizi başına düşün.* Kaçınılmaz döngüler (özyinelemeli IIR, PLL, Viterbi) için `scipy.signal.lfilter` gibi hazır C uygulamalarını ya da `numba` kullan.

1.3 Rehber Boyunca Kullanılan Ortak Başlık

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal as sig
4 from scipy import fft as spfft
5
6 # Grafiklerde tekrar eden kalibi tek yerde toplayalım.
7 def spektrum_ciz(x, fs, pencere='hann', db=True, eksen=None):
8     """Gerçek-değerli x işaretinin genlik spektrumunu cizer."""
9     N = len(x)
10    w = sig.get_window(pencere, N)
11    X = np.fft.rfft(x*w)
12    f = np.fft.rfftfreq(N, 1/fs)
13    mag = np.abs(X)/np.sum(w)*2 # tek yanlı, pencere kazancı telafili
14    y = 20*np.log10(np.maximum(mag, 1e-12)) if db else mag
15    ax = eksen or plt.gca()
16    ax.plot(f, y, lw=1.2, color='#2E6DA4')
17    ax.set_xlabel('Frekans (Hz)')
18    ax.set_ylabel('Genlik (dB)' if db else 'Genlik')
19    ax.grid(alpha=0.3)
20    return ax

```

2. İşaret Nedir? Analogdan Sayısala

Bir **işaret** (signal), bilgi taşıyan ve zamana (veya uzaya) göre değişen bir büyüklüktür. Mikrofonun hava basıncını gerilime çevirmesi, antenin elektromanyetik alanı akıma çevirmesi, sıcaklık sensörünün ısıyı volta çevirmesi — hepsi işaret üretir. İşaret işleme; bu büyüklükleri *ölçmek, dönüştürmek, filtrelemek, analiz etmek ve yeniden üretmek* sanatıdır.

	Zaman eksenı	Genlik eksenı
Analog	sürekli ($t \in \mathbb{R}$)	sürekli (her değeri alabilir)
Ayrık-zamanlı	kesikli ($n \in \mathbb{Z}$)	sürekli
Sayısal	kesikli	kesikli (kuantalanmış)

Gerçek dünya analogdur, bilgisayar sayısaldır. Köprü iki bileşendir: **ADC** analog işareti belirli aralıklarla ölçüp sayıya çevirir, **DAC** sayı dizisini tekrar sürekli bir gerilime çevirir.

Bilgi

DSP'nin bütün gücü şu basit gerçekten gelir: bir işareti sayı dizisine çevirdiğimizde onun üzerinde *aritmetik* yapabiliriz. Filtreleme bir çarpma-toplama, modülasyon bir çarpma, frekans analizi bir matris dönüşümü hâline gelir. Analogda pahalı devrelerle yapılan her şey, sayısalda birkaç satır koda dönüşür.

2.1 Ayrık-Zamanlı İşaretin NumPy'deki Karşılığı

Ayrık işareti $x[n]$ ile gösteririz; n tamsayı örnek indeksidir. Örnekler T_s saniye aralıkla alınıyorsa **örnekleme frekansı** $f_s = 1/T_s$ 'dir ve n 'inci örnek $t = n/f_s$ anına karşılık gelir.

```

1 import numpy as np
2
3 fs = 48_000          # ornekleme frekansi (Hz)
4 sure = 0.01         # saniye
5 f0 = 1_000.0        # sinus frekansi (Hz)
6
7 n = np.arange(int(sure*fs))      # ornek indeksleri: 0,1,2,...
8 t = n/fs                        # gercek zaman eksenı (s)
9 x = 0.8*np.sin(2*np.pi*f0*t)    # x[n] = A sin(2 pi f0 n/fs)
10
11 print(x.dtype, x.shape, x[:4])
12 # float64 (480,) [0. 0.13053 0.25882 0.38268]
```

Sayısal Açısal Frekans

$$\omega = 2\pi \frac{f}{f_s} \quad [\text{radyan/örnek}], \quad x[n] = A \sin(\omega n + \varphi).$$

DSP'de neredeyse her formül bu normalize ω cinsinden yazılır; çünkü algoritma açısından önemli olan mutlak frekans değil, örnekleme hızına *oranıdır*. $\omega = \pi$ tam olarak Nyquist frekansına karşılık gelir.

2.2 Veri Tipleri: Bellek ve Hız

dtype	Kullanım
float64	NumPy varsayılanı. Analiz ve prototip için ideal.
float32	Yarı bellek, çoğu SDR hattı için fazlasıyla yeterli (24 bit mantis).
complex64	IQ örnekleri (2×float32). GNU Radio'nun standart tipi.
complex128	Yüksek hassasiyetli karmaşık işlemler; iki kat bellek.
int16	WAV dosyaları ve ses donanımı; ±32767 aralığı.
uint8	RTL-SDR ham çıktısı; 0–255, merkez 127.5.

```

1 x64 = np.sin(2*np.pi*1000*np.arange(48000)/48000)      # float64: 384 KB
2 x32 = x64.astype(np.float32)                            # float32: 192 KB
3 iq  = (x32 + 1j*x32).astype(np.complex64)               # complex64: 384 KB
4
5 print(x64.nbytes, x32.nbytes, iq.nbytes)                 # 384000 192000 384000

```

İpucu

Uzun IQ akışlarında complex64 kullan: bellek yarıya iner, FFT neredeyse iki kat hızlanır ve hassasiyet kaybı (yaklaşık −140 dB) hiçbir radyo uygulamasında fark edilmez.

3. Örnekleme, Nyquist ve Aliasing

DSP'nin en temel teoremi **Nyquist–Shannon örnekleme teoremidir**: en yüksek frekans bileşeni $f_{\max}$ olan bir işareti kayıpsız temsil etmek için

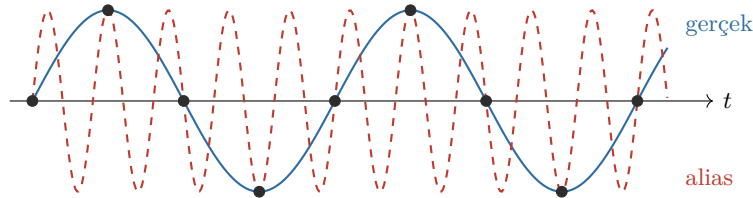
$$f_s > 2f_{\max}$$

olmalıdır. $f_s/2$ değerine **Nyquist frekansı** denir; sayısal dünyada temsil edebileceğimiz en yüksek frekanstır.

3.1 Aliasing: Kimlik Değiştiren Frekanslar

Nyquist kriterini ihlal edersek yüksek frekanslar düşük frekanslar gibi *kılık değiştirir*. Matematiksel olarak f ile $f + kf_s$ frekansları (her tamsayı k için) örneklendiğinde *tamamen aynı* dizi üretir:

$$\sin(2\pi(f + kf_s)n/f_s) = \sin(2\pi fn/f_s + 2\pi kn) = \sin(2\pi fn/f_s).$$



Örnekler (siyah noktalar) iki işareti ayırt edemez

```

1 import numpy as np
2
3 fs = 1000.0      # ornekleme frekansi
4 f1, f2 = 300.0, 700.0  # 700 = fs - 300 -> ayni ornekler

```

```

5
6 n = np.arange(10)
7 x1 = np.sin(2*np.pi*f1*n/fs)
8 x2 = np.sin(2*np.pi*f2*n/fs)
9
10 print(np.max(np.abs(x1 + x2))) # ~1e-15 -> x2 = -x1 (isaret ters, aynı frekans)
11
12 # Genel alias kuralı: gözlenen frekans
13 def alias_frekuensi(f, fs):
14     """f frekansının fs ile orneklendiğinde görüneceği frekans (0..fs/2)."""
15     f_mod = np.mod(f, fs)
16     return np.where(f_mod > fs/2, fs - f_mod, f_mod)
17
18 for f in [100, 400, 600, 900, 1100, 1400]:
19     print(f"{f:>5} Hz -> görünen {alias_frekuensi(f, 1000.0):>5.0f} Hz")

```

```

100 Hz -> görünen 100 Hz
400 Hz -> görünen 400 Hz
600 Hz -> görünen 400 Hz
900 Hz -> görünen 100 Hz
1100 Hz -> görünen 100 Hz
1400 Hz -> görünen 400 Hz

```

Dikkat

Aliasing **geri döndürülemez**. Örnekleme yapıldıktan sonra hangi frekansın hangi aliası olduğunu ayırt etmenin yolu yoktur. Çözüm, ADC'den *önce* analog bir **anti-aliasing** alçak geçiren filtre koymaktır. Aynı şey sayısal desimasyonda da geçerlidir: hız düşürmeden önce mutlaka filtrele (Kısım 4).

3.2 Aliasing'i Duymak

Aliasing'i en iyi kulakla anlarsın: frekansı sürekli artan bir chirp'i düşük f_s ile üretirsen, ses Nyquist'e ulaştığında "geri döner".

```

1 import numpy as np, soundfile as sf
2
3 fs = 8000
4 t = np.arange(int(3*fs))/fs
5 # 0 Hz'den 8000 Hz'e giden chirp: Nyquist (4000) sonrası geri katlanır
6 faz = 2*np.pi*np.cumsum(np.linspace(0, 8000, len(t)))/fs
7 x = 0.4*np.sin(faz)
8 sf.write('alias_chirp.wav', x.astype(np.float32), fs)
9
10 # Karsılastırma için doğru yol: önce yüksek fs'te uret, sonra filtreleyip indir
11 fs_hi = 48000
12 t_hi = np.arange(int(3*fs_hi))/fs_hi
13 faz_hi = 2*np.pi*np.cumsum(np.linspace(0, 8000, len(t_hi)))/fs_hi
14 x_hi = 0.4*np.sin(faz_hi)
15 from scipy import signal as sig
16 x_ok = sig.resample_poly(x_hi, up=1, down=6) # anti-alias filtre dahil
17 sf.write('alias_yok.wav', x_ok.astype(np.float32), fs)

```

Tekerlek Etkisi

Filmlerde araba tekerleklerinin geriye dönüyormuş gibi görünmesi tam olarak aliasing'dir: kamera saniyede 24 kare örnekler, teker dönüş frekansı Nyquist'i (12 devir/s) aşınca alias frekansı negatif çıkar ve teker geriye dönüyor görünür.

4. Kuantalama ve ADC

Örnekleme zamanı kesikli yapar; **kuantalama** genliği kesikli yapar. b bitlik bir ADC, giriş aralığını 2^b seviyeye böler.

Kuantalama Gürültüsü ve SQNR

Adım büyüklüğü $\Delta = \frac{V_{pp}}{2^b}$ ise kuantalama hatası $[-\Delta/2, \Delta/2]$ aralığında düzgün dağılır ve gücü $\sigma_q^2 = \Delta^2/12$ 'dir. Tam ölçekli sinüs için

$$\text{SQNR [dB]} = 6,02b + 1,76.$$

Yani her bit yaklaşık 6 dB dinamik aralık kazandırır.

```
1 import numpy as np
2
3 def kuantala(x, bit, vmax=1.0):
4     """x'i +-vmax aralığında bit-bitlik seviyelere yuvarlar (mid-tread)."""
5     adim = 2*vmax/2**bit
6     return np.clip(np.round(x/adim)*adim, -vmax, vmax - adim)
7
8 fs, f0 = 48000, 1000.0
9 t = np.arange(fs)/fs
10 x = 0.999*np.sin(2*np.pi*f0*t)
11
12 for b in [4, 8, 12, 16]:
13     xq = kuantala(x, b)
14     hata = xq - x
15     sqnr = 10*np.log10(np.mean(x**2)/np.mean(hata**2))
16     print(f"{b:>2} bit   SQNR = {sqnr:6.2f} dB   (teorik {6.02*b + 1.76:6.2f} dB)")
```

4 bit	SQNR = 24.40 dB	(teorik 25.84 dB)
8 bit	SQNR = 49.59 dB	(teorik 49.92 dB)
12 bit	SQNR = 74.40 dB	(teorik 74.00 dB)
16 bit	SQNR = 97.24 dB	(teorik 98.08 dB)

4.1 Dither: Gürültü Ekleyerek Kaliteyi Artırmak

Çok düşük seviyeli işaretlerde kuantalama hatası işaretle *ilişkilidir* ve harmonik bozulma (çirkin, “kırık” bir ses) üretir. Kuantalamadan önce çok küçük bir gürültü (*dither*) eklemek bu hatayı ilişkisiz, düz bir gürültü tabanına çevirir — toplam gürültü artar ama bozulma duyulmaz olur.

```
1 gurultu = np.random.uniform(-0.5, 0.5, len(x))*(2/2**8)    # 1 LSB genisliginde
2 x_dither = kuantala(x + gurultu, 8)
```

Bilgi

Aynı fikir *gürültü şekillendirme* (noise shaping) ile birleşince kuantalama gürültüsünü duyulmayan yüksek frekanslara itmeyi sağlar; CD masteringinde ve $\Sigma\Delta$ (sigma-delta) ADC'lerde kullanılan yöntem budur.

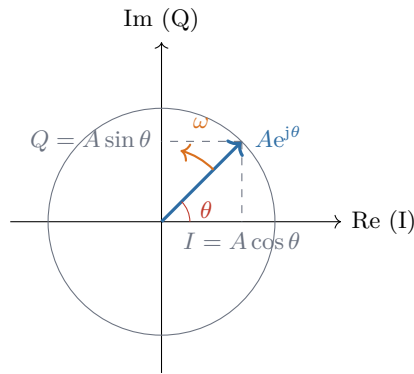
5. Karmaşık Sayılar ve Fazörler

DSP'de karmaşık sayılar süs değil, zorunluluktur: bir sinüsün hem *genliğini* hem *fazını* tek bir sayıda taşırlar ve negatif frekansları temsil etmeyi mümkün kılarlar.

Euler Formülü

$$e^{j\theta} = \cos \theta + j \sin \theta, \quad \cos \theta = \frac{e^{j\theta} + e^{-j\theta}}{2}, \quad \sin \theta = \frac{e^{j\theta} - e^{-j\theta}}{2j}.$$

Bir *fazör* $Ae^{j(\omega n + \varphi)}$, karmaşık düzlemde sabit hızla dönen bir vektördür: A yarıçap, ω dönme hızı, φ başlangıç açısıdır.



```

1 import numpy as np
2
3 z = 3 + 4j
4 print(abs(z), np.angle(z), np.angle(z, deg=True)) # 5.0 0.9273 rad 53.13 deg
5 print(z.real, z.imag, np.conj(z)) # 3.0 4.0 (3-4j)
6
7 # Fazör üretme: en sık kullanacağın tek satır
8 fs, f0, N = 48000, 1000.0, 480
9 n = np.arange(N)
10 faz = np.exp(2j*np.pi*f0*n/fs) # e^{j 2 \pi f_0 n / f_s}
11
12 # Önemli özellikler
13 print(np.allclose(np.abs(faz), 1.0)) # True -> birim cemberde
14 print(np.allclose(faz.real, np.cos(2*np.pi*f0*n/fs))) # True

```

5.1 Neden Negatif Frekans Var?

Gerçek bir kosinüs, zıt yönde dönen *iki* fazörün toplamıdır:

$$\cos(\omega n) = \frac{1}{2}e^{j\omega n} + \frac{1}{2}e^{-j\omega n}.$$

Bu yüzden gerçek işaretlerin spektrumu her zaman simetriktir ($X[-f] = X^*[f]$) ve bilginin yarısı gereksizdir. Karmaşık (IQ) işaretlerde ise pozitif ve negatif frekanslar *bağımsızdır*; bu da aynı örnek-leme hızında iki kat bant genişliği taşımayı sağlar. Kısım 4'ün tamamı bu fikrin üzerine kuruludur.

```

1 N, fs = 1024, 1000.0
2 n = np.arange(N)
3
4 gercek = np.cos(2*np.pi*100*n/fs)      # spektrumda +100 ve -100
5 karmasik = np.exp(2j*np.pi*100*n/fs)  # spektrumda yalnızca +100
6
7 for ad, x in [('gercek', gercek), ('karmasik', karmasik)]:
8     X = np.fft.fftshift(np.abs(np.fft.fft(x)))
9     f = np.fft.fftshift(np.fft.fftfreq(N, 1/fs))
10    tepe = f[np.argsort(X)[-2:]]        # en büyük iki tepe
11    print(f"{ad:>9}: tepeler {np.sort(tepe)}")
12 # gercek: [-100.  100.]      karmasik: [ 99.6 100. ] -> tek tepe

```

6. Sinyal Üretimi ve WAV Dosyaları

6.1 Temel Dalga Biçimleri

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs = 48000
5 t = np.arange(int(1.0*fs))/fs
6
7 sinus = np.sin(2*np.pi*440*t)
8 kare = sig.square(2*np.pi*440*t, duty=0.5)
9 testere = sig.sawtooth(2*np.pi*440*t)
10 ucgen = sig.sawtooth(2*np.pi*440*t, width=0.5)
11 chirp = sig.chirp(t, f0=100, f1=8000, t1=1.0, method='linear')
12 darbe = sig.unit_impulse(len(t), 'mid')
13 beyaz = np.random.default_rng(0).normal(0, 0.1, len(t))

```

Dikkat

Kare ve testere dalgaları *sonsuz* harmonik içerir; doğrudan üretilirse Nyquist üstündeki harmonikler alias olur ve ses kirlenir. Temiz sentez için ya bant-sınırlı üretim (BLIT/PolyBLEP) ya da yüksek hızda üretilip `resample_poly` ile indirme kullanılır.

6.2 WAV Okuma ve Yazma

```

1 import soundfile as sf
2 import numpy as np
3
4 # Yazma: float32 [-1, 1] aralığında; kirpilmayı (clipping) onle
5 x = 0.5*np.sin(2*np.pi*440*np.arange(48000)/48000)
6 sf.write('la440.wav', x.astype(np.float32), 48000)
7
8 # Okuma: her zaman fs'i de al
9 y, fs = sf.read('la440.wav', dtype='float32')
10 print(y.shape, fs, y.dtype)      # (48000,) 48000 float32

```

```

11
12 # Stereo: (N, 2) sekilli dizi
13 stereo = np.stack([x, np.roll(x, 100)], axis=1)
14 sf.write('stereo.wav', stereo.astype(np.float32), 48000)

1 # scipy alternatifini (ek bağımlılık istemiyorsan)
2 from scipy.io import wavfile
3
4 wavfile.write('int16.wav', 48000, (x*32767).astype(np.int16))
5 fs2, z = wavfile.read('int16.wav')
6 z = z.astype(np.float32)/32768.0      # normalize etmeyi UNUTMA

```

İpucu

scipy.io.wavfile okuduğunda dosyadaki ham tipi (int16) verir; ± 1 aralığına *sen* normalize etmelisin. soundfile ise dtype='float32' ile bunu otomatik yapar — bu yüzden rehber boyunca soundfile tercih edilir.

6.3 Gerçek Zamanlı Ses Girişi/Çıkışı

```

1 import sounddevice as sd
2 import numpy as np
3
4 fs = 48000
5 # Cal
6 sd.play(0.3*np.sin(2*np.pi*440*np.arange(fs)/fs), fs); sd.wait()
7
8 # Kaydet (2 saniye, mono)
9 kayit = sd.rec(int(2*fs), samplerate=fs, channels=1, dtype='float32')
10 sd.wait()
11 kayit = kayit[:, 0]
12
13 # Akis (callback) ile gercek zamanli isleme
14 def callback(indata, outdata, frames, time, status):
15     outdata[:] = 0.5*indata      # burada filtre/demodulasyon yapilir
16
17 with sd.Stream(samplerate=fs, blocksize=1024, channels=1, callback=callback):
18     sd.sleep(5000)      # 5 saniye calis

```

Dikkat

Callback fonksiyonu **gerçek zamanlı** bir bağlamda çalışır: içinde bellek ayırma, dosya G/Ç, print veya uzun hesap yapma. Aksi hâlde ses kesilir (underrun). Ağır işi bir kuyruğa (queue.Queue) atıp başka bir iş parçacığında yap.

KISIM 2

Frekans Alanı: DFT ve FFT

7. Fourier Fikri

Fourier'in fikri tek cümleyle özetlenir: **her işaret, farklı frekanslardaki sinüzoidlerin toplamı olarak yazılabilir.** Zaman alanında karmaşık görünen bir problem (örneğin “bu sesteki uğultuyu at”) frekans alanında basitleşir (“50 Hz bileşenini sil”).

Ayrık Fourier Dönüşümü (DFT)

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{j2\pi kn/N}.$$

$X[k]$, işaretin $f_k = k f_s/N$ frekansındaki bileşenin genlik ve fazını taşıyan karmaşık sayıdır.

DFT'nin sezgisi bir *iç çarpımdır*: işareti, her frekanstaki bir referans fazörle çarpıp toplarız. İşarete o frekans varsa terimler birbirini destekler ve büyük bir sayı çıkar; yoksa fazlar rastgele dağılır ve toplam sıfıra yakın olur.

7.1 DFT'yi Doğrudan Yazmak

```
1 import numpy as np
2
3 def dft_yavas(x):
4     """Tanimdan DFT: O(N^2). Ogrenmek icin; uretimde asla kullanma."""
5     x = np.asarray(x, dtype=complex)
6     N = len(x)
7     n = np.arange(N)
8     k = n.reshape((N, 1))
9     W = np.exp(-2j*np.pi*k*n/N)          # NxN donusum matrisi
10    return W @ x
11
12 x = np.random.default_rng(0).normal(size=64)
13 print(np.allclose(dft_yavas(x), np.fft.fft(x)))    # True
```

Dikkat

Matris yöntemi N^2 karmaşık sayı saklar: $N = 10^5$ için 160 GB bellek. Öğretici olarak güzeldir, pratikte kullanılamaz. Gerçek hayatta her zaman `np.fft` (veya `scipy.fft`) kullan — $O(N \log N)$.

7.2 Frekans Eksenini ve Ölçekleme

En sık yapılan hata FFT çıktısını doğru frekansa ve doğru genliğe eşleyememektir.

Fonksiyon	Ne verir?
<code>np.fft.fft(x)</code>	Karmaşık spektrum, N nokta, $0 \dots f_s$ sırasıyla.
<code>np.fft.rfft(x)</code>	Gerçek girdi için yalnız pozitif frekanslar, $N/2 + 1$ nokta ($2\times$ hızlı).
<code>np.fft.fftfreq(N, 1/fs)</code>	Frekans eksenini; ikinci yarısı negatif frekanslar.
<code>np.fft.rfftfreq(N, 1/fs)</code>	$0 \dots f_s/2$ arası frekans eksenini.
<code>np.fft.fftshift(X)</code>	Sıfır frekansı ortaya alır (IQ spektrumları için şart).
<code>np.fft.ifft / irfft</code>	Ters dönüşüm.

Genlik Ölçekleme

Genliği A olan bir sinüs için FFT tepesi $NA/2$ büyüklüğündedir. Doğru genliği geri almak için:

$$\text{tek yanlı genlik} = \frac{2|X[k]|}{N} \quad (k = 0 \text{ ve } k = N/2 \text{ için } 2 \text{ çarpanı yok}).$$

Pencere kullanılıyorsa N yerine *pencerenin toplamı* $\sum w[n]$ ile böl.

```

1 import numpy as np
2
3 fs, N = 1000.0, 1000
4 n = np.arange(N)
5 x = 2.5*np.sin(2*np.pi*100*n/fs) + 1.0*np.sin(2*np.pi*250*n/fs)
6
7 X = np.fft.rfft(x)
8 f = np.fft.rfftfreq(N, 1/fs)
9 genlik = 2*np.abs(X)/N
10
11 for hedef in [100, 250]:
12     k = np.argmax(np.abs(f - hedef))
13     print(f"{f[k]:6.1f} Hz -> genlik {genlik[k]:.3f}")

```

```

100.0 Hz -> genlik 2.500
250.0 Hz -> genlik 1.000

```

7.3 Çözünürlük, Sızıntı ve Sıfır Doldurma

Frekans Çözünürlüğü

$$\Delta f = \frac{f_s}{N} = \frac{1}{T}$$

Burada $T = N/f_s$ gözlem süresidir. Çözünürlüğü artırmanın tek yolu daha uzun süre gözlemektir; sıfır doldurma (zero-padding) çözünürlük *kazandırmaz*, sadece spektrumu daha sık örnekler (interpolasyon).

```

1 fs = 1000.0
2 for N in [100, 1000, 10000]:

```

```

3     print(f"N={N:>6}  T={N/fs:6.2f} s  df={fs/N:6.2f} Hz")
4     # N=   100  T=  0.10 s  df= 10.00 Hz
5     # N=  1000  T=  1.00 s  df=  1.00 Hz
6     # N= 10000  T= 10.00 s  df=  0.10 Hz
7
8     # Sifir doldurma: tepe konumunu daha iyi okumak için
9     x = np.sin(2*np.pi*100.5*np.arange(256)/fs)
10    X1 = np.abs(np.fft.rfft(x))           # 129 nokta
11    X2 = np.abs(np.fft.rfft(x, n=4096))   # 2049 nokta, aynı bilgi

```

Dikkat

FFT, işaretin *periyodik* olduğunu varsayar. Pencere sınırında işaret kendine düzgün eklenmiyorsa (frekans, bin merkezine denk gelmiyorsa) kesme süresizliği spektrumda **sızıntı** (spectral leakage) üretir: tek bir sinüs, tüm spektruma yayılır. Çözüm pencerelemedir.

8. FFT: Hızlı Fourier Dönüşümü

FFT bir dönüşüm değil, DFT'yi hesaplamamanın hızlı bir *algoritmasıdır*. Temel fikir böl-ve-fethet: N noktalı DFT, çift ve tek indisli örneklerin $N/2$ noktalı iki DFT'sine indirgenir.

Radix-2 Kelebek (Butterfly)

$$X[k] = E[k] + e^{-j2\pi k/N} O[k], \quad X[k + N/2] = E[k] - e^{-j2\pi k/N} O[k]$$

E : çift indisli örneklerin DFT'si, O : tek indislilerin. Karmaşıklık $O(N^2) \rightarrow O(N \log N)$ 'e iner.

```

1  import numpy as np
2
3  def fft_radix2(x):
4      """Özyinelemeli radix-2 FFT. N ikinin kuvveti olmalı."""
5      x = np.asarray(x, dtype=complex)
6      N = len(x)
7      if N & (N - 1):
8          raise ValueError("N ikinin kuvveti olmalı")
9      if N == 1:
10         return x
11     cift = fft_radix2(x[0::2])
12     tek  = fft_radix2(x[1::2])
13     W = np.exp(-2j*np.pi*np.arange(N//2)/N)   # twiddle carpanlari
14     return np.concatenate([cift + W*tek, cift - W*tek])
15
16 x = np.random.default_rng(1).normal(size=1024)
17 print(np.allclose(fft_radix2(x), np.fft.fft(x)))   # True

```

8.1 Yerinde (In-Place) Yinelemeli FFT

Özyinelemeli sürüm her adımda yeni dizi ayırır. Üretim uygulamaları *yerinde* çalışır: önce girdiyi bit-ters (bit-reversal) sıraya dizer, sonra $\log_2 N$ kademede kelekleri uygular.

```

1  import numpy as np
2
3  def bit_ters_permutasyon(N):
4      """0..N-1 için bit-ters indeks tablosu (N ikinin kuvveti)."""

```

```

5     bit = int(np.log2(N))
6     idx = np.arange(N)
7     ters = np.zeros(N, dtype=int)
8     for b in range(bit):
9         ters |= ((idx >> b) & 1) << (bit - 1 - b)
10    return ters
11
12    def fft_yerinde(x):
13        """Yinelemeli radix-2 FFT: kademe kademe, ek dizi ayırmadan."""
14        x = np.asarray(x, dtype=complex).copy()
15        N = len(x)
16        if N & (N - 1):
17            raise ValueError("N ikinin kuvveti olmalı")
18
19        x = x[bit_ters_permutasyon(N)]          # 1) bit-ters sıralama
20
21        adim = 2
22        while adim <= N:                        # 2) log2(N) kademe
23            W = np.exp(-2j*np.pi*np.arange(adim//2)/adim)
24            for bas in range(0, N, adim):
25                ust = x[bas:bas + adim//2].copy()
26                alt = x[bas + adim//2:bas + adim]*W
27                x[bas:bas + adim//2] = ust + alt    # kelebek
28                x[bas + adim//2:bas + adim] = ust - alt
29            adim *= 2
30        return x
31
32    x = np.random.default_rng(3).normal(size=2048)
33    print(np.allclose(fft_yerinde(x), np.fft.fft(x)))    # True

```

Bilgi

Bit-ters sıralamanın nedeni, böl-ve-fethetin çift/tek ayrımını tersten uygulamaktır: özyinelemede “önce böl, sonra birleştir” yapılan iş, yinelemeli sürümde “önce doğru sıraya diz, sonra birleştir” hâline gelir. Bu, FFTW ve pocketfft gibi kütüphanelerin de temel yapısıdır — üstüne SIMD, önbellek blokları ve karışık radix eklenmiştir.

8.2 Hız Karşılaştırması

```

1    import numpy as np, time
2
3    x = np.random.default_rng(0).normal(size=4096)
4
5    def sure_olc(fn, tekrar=20):
6        t0 = time.perf_counter()
7        for _ in range(tekar):
8            fn(x)
9        return (time.perf_counter() - t0)/tekar*1e3    # ms
10
11    print(f"dft_yavas      : {sure_olc(dft_yavas, 3):8.2f} ms")
12    print(f"fft_radix2     : {sure_olc(fft_radix2):8.2f} ms")
13    print(f"np.fft.fft      : {sure_olc(np.fft.fft):8.2f} ms")
14    print(f"np.fft.rfft     : {sure_olc(np.fft.rfft):8.2f} ms")

```

```

dft_yavas      :    210.44 ms
fft_radix2     :     12.83 ms

```

```
np.fft.fft :    0.06 ms
np.fft.rfft :    0.03 ms
```

İpucu

Kütüphane FFT'si kendi yazdığından yüzlerce kat hızlıdır: SIMD, önbellek dostu erişim ve karışık radix (2,3,5,7) desteği vardır. `scipy.fft` ayrıca `workers=-1` ile çok çekirdek kullanır. Uzunluğu `next_fast_len` ile yuvarlamak, asal uzunluklu FFT'lerdeki yavaşlığı önler.

```
1 from scipy import fft as spfft
2
3 N = 10007                                # asal uzunluk: yavas
4 Nf = spfft.next_fast_len(N)              # 10080 gibi 2/3/5-carpanli sayi
5 X = spfft.fft(x, n=Nf, workers=-1)      # cok cekirdekli
```

9. Pencereleme (Windowing)

Pencereleme, kesme süreksizliğini yumuşatarak sızıntıyı azaltır. Bedeli, ana lobun genişlemesi — yani *çözünürlük* ile *dinamik aralık* arasında bir takas yaparız.

Pencere	Yan lob	Ana lob	Ne zaman?
boxcar	-13 dB	1,0 bin	En iyi çözünürlük, en kötü sızıntı.
hann	-31 dB	2,0 bin	Genel amaçlı varsayılan.
hamming	-43 dB	2,0 bin	İlk yan lobu bastırır.
blackman	-58 dB	3,0 bin	Yüksek dinamik aralık.
blackmanharris	-92 dB	4,0 bin	Çok zayıf işaretleri görmek için.
kaiser(β)	ayarlanır	ayarlanır	β ile takası sen seç.
flattop	-88 dB	4,7 bin	Genlik ölçümü (tepe hatası < 0,01 dB).

```
1 import numpy as np
2 from scipy import signal as sig
3
4 fs, N = 1000.0, 1024
5 n = np.arange(N)
6 # Bin merkezine denk GELMEYEN bir frekans: sızıntı görünür
7 x = np.sin(2*np.pi*100.5*n/fs) + 1e-4*np.sin(2*np.pi*250*n/fs) # -80 dB komşu
8
9 for ad in ['boxcar', 'hann', 'blackmanharris']:
10     w = sig.get_window(ad, N)
11     X = np.fft.rfft(x*w)
12     mag = 20*np.log10(np.abs(X)/np.sum(w)*2 + 1e-15)
13     f = np.fft.rfftfreq(N, 1/fs)
14     k = np.argmin(np.abs(f - 250))
15     print(f"{ad:>15}: 250 Hz'te okunan seviye = {mag[k]:7.2f} dB")
```

```
boxcar: 250 Hz'te okunan seviye = -61.51 dB
hann: 250 Hz'te okunan seviye = -80.00 dB
blackmanharris: 250 Hz'te okunan seviye = -79.99 dB
```

Dikdörtgen pencerede -80 dB'lik zayıf işaret, güçlü komşunun sızıntısı altında kalıyor ve 18 dB yanlış (çok güçlü) okunuyor. Hann ve Blackman-Harris ile seviye doğru çıkıyor — pencereleme burada bir süsleme değil, ölçümün doğru olup olmamasının farkı.

Pencere Kazançları

İki farklı normalizasyon vardır ve karıştırılırsa sonuç birkaç dB kayar:

$$\text{tutarlı kazanç (genlik için)} = \frac{1}{N} \sum_n w[n], \quad \text{güç kazancı (PSD için)} = \frac{1}{N} \sum_n w^2[n].$$

10. Güç Spektral Yoğunluğu ve Spektrogram

Tek bir FFT gürültülü bir kestirimdir: varyansı, veri uzunluğundan bağımsız olarak ortalamanın kendisi kadardır. **Welch yöntemi**, işareti örtüşen bloklara böler, her birinin periyodogramını alır ve ortalar — gürültü tabanı düzleşir.

```
1 import numpy as np
2 from scipy import signal as sig
3 import matplotlib.pyplot as plt
4
5 fs = 48000
6 t = np.arange(4*fs)/fs
7 x = (np.sin(2*np.pi*1000*t) + 0.01*np.sin(2*np.pi*3000*t)
8     + np.random.default_rng(0).normal(0, 0.05, len(t)))
9
10 # Periyodogram (tek FFT) vs Welch (ortalama)
11 f1, P1 = sig.periodogram(x, fs, window='hann', scaling='density')
12 f2, P2 = sig.welch(x, fs, nperseg=4096, noverlap=2048, scaling='density')
13
14 plt.semilogy(f1, P1, lw=0.5, alpha=0.5, label='periyodogram')
15 plt.semilogy(f2, P2, lw=1.5, label='Welch')
16 plt.xlim(0, 6000); plt.legend(); plt.grid(alpha=0.3)
17 plt.xlabel('Hz'); plt.ylabel('PSD (V^2/Hz)'); plt.show()
```

Bilgi

`scaling='density'` sonucu V^2/Hz olarak verir (gürültü tabanı karşılaştırmaları için); `scaling='spectrum'` ise V^2 verir (ayrık tonların gücünü okumak için). Toplam gücü doğrulamak istersen Parseval'i kullan: $\text{np.trapz}(P, f) \approx \text{np.mean}(x**2)$.

10.1 Spektrogram: Zaman-Frekans Görüntüsü

```
1 f, tt, Sxx = sig.spectrogram(x, fs, window='hann', nperseg=1024,
2                             noverlap=768, scaling='density', mode='psd')
3 plt.pcolormesh(tt, f, 10*np.log10(Sxx + 1e-15), shading='gouraud',
4               cmap='magma', vmin=-120, vmax=-20)
5 plt.ylabel('Frekans (Hz)'); plt.xlabel('Zaman (s)')
6 plt.colorbar(label='dB'); plt.ylim(0, 6000); plt.show()
```

Zaman–Frekans Takası

Blok uzunluğu M için

$$\Delta f = \frac{f_s}{M}, \quad \Delta t = \frac{M}{f_s} \implies \Delta f \cdot \Delta t = 1.$$

Frekansta netlik istiyorsan zamanda bulanıklaşırın; bu, DSP’nin belirsizlik ilkesidir. Konuşma için ~ 25 ms, radyo sinyalleri için ~ 1 – 10 ms tipik seçimlerdir.

10.2 IQ İşaretlerin Spektrumu

Karmaşık işaretlerde spektrum simetrik *değildir*; negatif frekanslar gerçek bilgi taşır. Bu yüzden `fftshift` ve `return_onesided=False` şarttır.

```

1 fs = 2.4e6                                # SDR ornekleme hizi
2 iq = ...                                  # complex64 dizi
3
4 f, P = sig.welch(iq, fs, nperseg=4096, return_onesided=False, scaling='density')
5 f = np.fft.fftshift(f); P = np.fft.fftshift(P)
6 plt.plot(f/1e6, 10*np.log10(P + 1e-20))
7 plt.xlabel('Ara frekans (MHz)'); plt.ylabel('dB'); plt.grid(alpha=0.3); plt.show()

```

Dikkat

Gerçek işaret için `rfft`, karmaşık işaret için `fft + fftshift` kullan. Karmaşık bir IQ akışına `rfft` uygularsan negatif frekans bilgisini sessizce kaybedersin — spektrum “doğru görünür” ama yanlıştır.

KISIM 3

Filtreler: FIR ve IIR

11. Konvolüsyon ve FIR Filtreler

Doğrusal ve zamanla değişmeyen (LTI) her sistem **konvolüsyonla** tanımlanır. Sistemin dürtü yanıtı $h[n]$ ise çıkış:

$$y[n] = \sum_k h[k] x[n-k] = (h * x)[n].$$

Konvolüsyon Teoremi

Zaman alanındaki konvolüsyon, frekans alanında *çarpma*dır:

$$y = h * x \iff Y(f) = H(f) X(f).$$

Bu yüzden filtreleme, spektrumu $H(f)$ ile şekillendirmek demektir.

11.1 NumPy/SciPy ile Konvolüsyon

Fonksiyon	Ne zaman?
<code>np.convolve(x,h)</code>	Kısa filtreler (<100 katsayı), tek seferlik işler.
<code>sig.convolve(x,h,method='auto')</code>	Uzunluğa bakıp doğrudan/FFT yöntemini kendi seçer.
<code>sig.fftconvolve(x,h)</code>	Uzun filtreler; $O(N \log N)$.
<code>sig.lfilter(b,a,x)</code>	Nedensel, durum taşıyabilen (blok blok) filtreleme.
<code>sig.oaconvolve(x,h)</code>	Çok uzun akışlar için overlap-add.
<code>sig.filtfilt(b,a,x)</code>	İleri-geri: sıfır faz , gecikme yok (çevrimdışı).

```

1 import numpy as np
2 from scipy import signal as sig
3
4 x = np.random.default_rng(0).normal(size=100_000)
5 h = sig.firwin(301, 0.2)
6
7 y1 = np.convolve(x, h, mode='same')           # yavaş ama basit
8 y2 = sig.fftconvolve(x, h, mode='same')       # çok daha hızlı
9 y3 = sig.filtfilt(h, 1.0, x)                  # nedensel, (len(h)-1)/2 gecikmeli

```

```

10
11 print(np.allclose(y1, y2))           # True

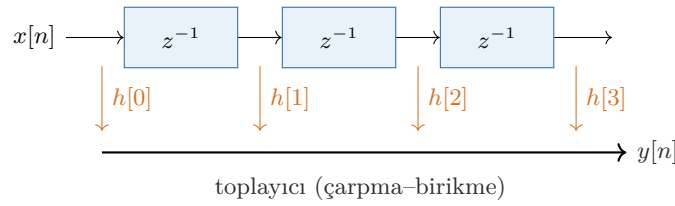
```

Dikkat

mode parametresi çıkış uzunluğunu belirler: 'full' ($N + M - 1$), 'same' (N , ortadan hizalı), 'valid' ($N - M + 1$, yalnız tam örtüşen kısım). Hizalama hatası, ölçümlerinde yarım filtre uzunluğu kadar gecikme olarak karşına çıkar.

11.2 FIR Filtrenin Doğrusal Fazı

FIR katsayıları simetrikse ($h[n] = h[M - 1 - n]$) filtre **doğrusal fazlıdır**: bütün frekanslar aynı miktarda gecikir, dalga biçimi bozulmaz. Gecikme tam olarak $(M - 1)/2$ örnektir.



11.3 Pencere Yöntemiyle Alçak Geçiren Tasarım

İdeal alçak geçiren filtrenin dürtü yanıtı bir *sinc*'tir; sonsuz uzun olduğu için kesip pencereleriz.

Kesilmiş Sinc

$$h[n] = 2f_c \text{sinc}(2f_c(n - M/2)) \cdot w[n], \quad f_c = \frac{f_{\text{kesim}}}{f_s}$$

```

1 import numpy as np
2 from scipy import signal as sig
3
4 def alcak_gecen_fir(kesim_hz, fs, taps=101, pencere='hamming'):
5     """Pencere yöntemiyle alcak gecirgen FIR (elle, ogretici)."""
6     fc = kesim_hz/fs                                # normalize kesim (0..0.5)
7     n = np.arange(taps) - (taps - 1)/2
8     h = 2*fc*np.sinc(2*fc*n)                        # np.sinc(x) = sin(pi x)/(pi x)
9     h *= sig.get_window(pencere, taps)
10    return h/np.sum(h)                               # DC kazanci 1 olsun
11
12 h_elle = alcak_gecen_fir(1000, 48000, 101)
13 h_scipy = sig.firwin(101, 1000, fs=48000, window='hamming')
14 print(np.allclose(h_elle, h_scipy))                # True

```

11.4 Filtreyi Doğrulamak: Frekans Yanıtı

```

1 import matplotlib.pyplot as plt
2
3 w, H = sig.freqz(h_scipy, worN=8192, fs=48000)
4
5 fig, (a1, a2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True)
6 a1.plot(w, 20*np.log10(np.abs(H) + 1e-12), color='#2E6DA4')
7 a1.set_ylabel('Genlik (dB)'); a1.set_ylim(-100, 5); a1.grid(alpha=0.3)

```

```

8 a1.axvline(1000, ls='--', color='#C0392B')
9 a2.plot(w, np.unwrap(np.angle(H)), color='#D9701E')
10 a2.set_ylabel('Faz (rad)'); a2.set_xlabel('Hz'); a2.grid(alpha=0.3)
11 plt.show()
12
13 # Grup gecikmesi: doğrusal fazlı FIR'de sabit olmalı
14 wg, gd = sig.group_delay(h_scipy, 1.0, fs=48000)
15 print(np.round(gd[:5], 3))      # [50. 50. 50. 50. 50.] -> (101-1)/2

```

11.5 Tasarım Parametreleri: Kaç Katsayı Gerekir?

Kaiser Tahmini

Geçiş bandı genişliği Δf ve durdurma bandı bastırması A dB için gereken katsayı sayısı yaklaşık

$$M \approx \frac{A - 8}{2,285 \cdot 2\pi \Delta f / f_s}.$$

Yani **keskin geçiş pahalıdır**: geçiş bandını yarıya indirmek katsayı sayısını iki katına çıkarır.

```

1 from scipy.signal import kaiserord, firwin, remez
2
3 fs = 48000
4 gecis = 500.0          # gecis bandi genisligi (Hz)
5 A = 60.0               # istenen bastirma (dB)
6
7 M, beta = kaiserord(A, gecis/(fs/2))
8 print(f"gereken katsayi: {M}, kaiser beta = {beta:.3f}")
9 h = firwin(M, 1000, window=('kaiser', beta), fs=fs)
10
11 # Alternatif: Parks-McClellan (esdalgalı, aynı bastırma için en az katsayı)
12 h_pm = remez(M, [0, 1000, 1500, fs/2], [1, 0], fs=fs)

```

İpucu

remez (Parks–McClellan) aynı şartname için pencere yönteminden %20–30 daha az katsayı kullanır; hesap yükünün kritik olduğu gerçek zamanlı hatlarda tercih edilir. Ama yakınsamayabilir — tasarımı her zaman freqz ile doğrula.

12. IIR Filtreler, SOS ve Sıfır Faz

IIR filtreler geri besleme kullanır: çok daha az katsayıyla keskin geçiş verirler, ama fazları doğrusal değildir ve kararsız olabilirler.

IIR Fark Denklemini

$$y[n] = \sum_{k=0}^P b_k x[n-k] - \sum_{k=1}^Q a_k y[n-k], \quad H(z) = \frac{b_0 + b_1 z^{-1} + \dots}{1 + a_1 z^{-1} + \dots}$$

Tip	Karakteri
Butterworth	Geçiş bandı maksimum düz; en yumuşak geçiş.
Chebyshev I	Geçiş bandında dalgalanma, daha keskin geçiş.
Chebyshev II	Durdurma bandında dalgalanma, geçiş bandı düz.
Elliptic (Cauer)	İki bantta da dalgalanma; verilen derece için en keskin.
Bessel	Neredeyse doğrusal faz; dalga biçimini korur (darbeler için).

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs = 48000
5 # HER ZAMAN sos (second-order sections) formati kullan: sayısal olarak kararlı
6 sos = sig.butter(8, 1000, btype='low', fs=fs, output='sos')
7 y = sig.sosfilt(sos, x) # nedensel (gerçek zamanlı)
8 y0 = sig.sosfiltfilt(sos, x) # ileri-geri: sıfır faz, 2x dik
9
10 # Bant geciren / bant durduran
11 sos_bp = sig.butter(6, [300, 3400], btype='bandpass', fs=fs, output='sos')
12 b_no, a_no = sig.iirnotch(50.0, Q=30, fs=fs) # sebeke ugultusu (b, a)
13
14 # Kararlılık ve kutup-sıfır kontrolü
15 z, p, k = sig.sos2zpk(sos)
16 print('en büyük kutup yarıçapı:', np.max(np.abs(p))) # < 1 olmalı

```

Dikkat

output='ba' (tek payda polinomu) yüksek dereceli filtrelerde **sayısal olarak patlar**: 8. dereceden sonra katsayılar arasındaki büyüklük farkı çift duyarlılığı aşar ve filtre kararsızlaşır. Her zaman output='sos' kullan ve sosfilt/sosfiltfilt ile uygula.

12.1 Sıfır Faz: filtfilt

filtfilt işareti önce ileri, sonra geri filtreler. Sonuç: faz bozulması tam olarak sıfır, genlik yanıtı ise karesi alınmış hâli (bastırma iki katı). Bedeli, nedensel olmamasıdır — yalnızca kayıtlı veride kullanılır.

```

1 y_ileri = sig.sosfilt(sos, x) # gecikmeli (grup gecikmesi kadar)
2 y_sifir = sig.sosfiltfilt(sos, x) # gecikmesiz, gerçek zamanda KULLANILAMAZ

```

12.2 Biquad: IIR'ın Yapı Taşı

```

1 def biquad_lowpass(f0, fs, Q=0.7071):
2     """Audio EQ Cookbook alçak gecirgen biquad katsayıları."""
3     w0 = 2*np.pi*f0/fs
4     alpha = np.sin(w0)/(2*Q)
5     cw = np.cos(w0)
6     b = np.array([(1 - cw)/2, 1 - cw, (1 - cw)/2])
7     a = np.array([1 + alpha, -2*cw, 1 - alpha])
8     return b/a[0], a/a[0]
9
10 b, a = biquad_lowpass(1000, 48000)
11 w, H = sig.freqz(b, a, worN=4096, fs=48000)

```

13. Ucuz Filtreler: Hareketli Ortalama, CIC, DC Engelleyici

13.1 Hareketli Ortalama

En basit alçak geçiren filtre: son M örneğin ortalaması. Kümülatif toplamla $O(1)$ maliyetle uygulanır.

```
1 def hareketli_ortalama(x, M):
2     """cumsum hilesiyle  $O(N)$  hareketli ortalama."""
3     c = np.cumsum(np.insert(x, 0, 0.0))
4     return (c[M:] - c[:-M])/M
5
6 # Frekans yaniti: sinc bicimli, ilk sifir  $fs/M$ 'de
7 M = 16
8 w, H = sig.freqz(np.ones(M)/M, 1.0, worN=4096, fs=48000)
```

Dikkat

Hareketli ortalamanın durdurma bandı bastırması yalnızca -13 dB'dir; yan lobları çok yüksektir. Ucuzdur ama "iyi filtre" değildir — seçicilik gerekiyorsa FIR/IIR kullan. Ard arda 3–5 kez uygulamak (kaskad) yan lobları belirgin biçimde bastırır.

13.2 CIC (Cascaded Integrator-Comb)

Çarpma içermeyen, yalnızca toplama-çıkarma kullanan desimasyon filtresi; FPGA ve SDR ön uçlarının vazgeçilmezi.

```
1 def cic_desimasyon(x, R, N=3):
2     """R kat desimasyon, N mertebeli CIC (carpma yok)."""
3     y = x.astype(np.float64)
4     for _ in range(N):                # integratorler
5         y = np.cumsum(y)
6     y = y[::R]                        # hiz dusurme
7     for _ in range(N):                # comb (fark) asamalari
8         y = np.diff(y, prepend=0.0)
9     return y/(R**N)                  # kazanc telafisi
```

13.3 DC Engelleyici

Ortalamayı (0 Hz bileşenini) süzen tek kutuplu yüksek geçiren:

$$y[n] = x[n] - x[n-1] + \alpha y[n-1], \quad \alpha \approx 0.995.$$

```
1 def dc_engelle(x, alpha=0.995):
2     return sig.lfilter([1, -1], [1, -alpha], x)
```

Bilgi

RTL-SDR gibi doğrudan dönüştürücü alıcılarda merkez frekansta güçlü bir **DC tepesi** bulunur (LO sızıntısı). Bu tepe demodülasyonu bozar; ya DC engelleyiciyle süzülür ya da ilgilenilen kanal biraz kaydırılarak (offset tuning) DC'den uzaklaştırılır.

14. Akış İşleme: Blok Blok Filtreleme

Gerçek zamanlı sistemlerde veri parça parça gelir. Filtre durumunu bloklar arasında taşımazsan her blok sınırında tıklama sesi duyarsın.

```

1 import numpy as np
2 from scipy import signal as sig
3
4 sos = sig.butter(6, 3000, fs=48000, output='sos')
5 zi = sig.sosfilt_zi(sos)*0.0          # baslangic durumu (sifir)
6
7 def blok_isle(blok):
8     global zi
9     y, zi = sig.sosfilt(sos, blok, zi=zi)    # durum tasinir
10    return y
11
12 # 1024'luk bloklarla akis
13 akis = np.random.default_rng(0).normal(size=48000)
14 cikis = np.concatenate([blok_isle(akis[i:i+1024])
15                          for i in range(0, len(akis), 1024)])
16
17 # Dogrulama: tek seferde filtrelemekle ayni sonuc
18 print(np.allclose(cikis, sig.sosfilt(sos, akis)))    # True

```

İpucu

FIR için aynı iş sig.lfilter(h, 1.0, blok, zi=zi) ile yapılır; uzun FIR'lerde sig.oaconvolve (overlap-add) daha hızlıdır. Overlap-add mantığı: her bloğu FFT ile filtrele, taşan $M - 1$ örneği bir sonraki bloğa ekle.

```

1 def overlap_add(x, h, blok=4096):
2     """Elle overlap-add: uzun FIR'i FFT ile blok blok uygular."""
3     M = len(h)
4     Nf = 1 << int(np.ceil(np.log2(blok + M - 1)))
5     H = np.fft.rfft(h, Nf)
6     y = np.zeros(len(x) + M - 1)
7     for i in range(0, len(x), blok):
8         parca = x[i:i+blok]
9         Y = np.fft.irfft(np.fft.rfft(parca, Nf)*H, Nf)
10        y[i:i+len(Y)] += Y[:len(y) - i]
11    return y[:len(x)]

```

KISIM 4

IQ ve Karmaşık Bant Temeli

15. IQ Nedir? Analitik İşaret

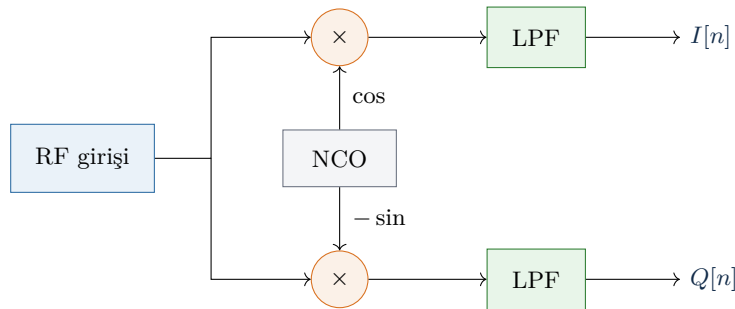
Modern telsiz haberleşmenin tamamı tek bir fikre dayanır: yüksek frekanslı gerçek bir işareti, *karmaşık* ve düşük hızlı bir işarete indirgemek. Bu karmaşık işarete **bant temeli** (baseband) ya da **IQ** denir.

IQ Gösterimi

$$s(t) = A(t) \cos(2\pi f_c t + \varphi(t)) = \Re \left\{ \underbrace{A(t)e^{j\varphi(t)}}_{\text{bant temeli } z(t)} e^{j2\pi f_c t} \right\}$$

$$z(t) = I(t) + jQ(t), \quad I = A \cos \varphi, \quad Q = A \sin \varphi, \quad A = |z|, \quad \varphi = \arg z.$$

Yani I ve Q , taşıyıcının genlik ve fazını taşıyan iki gerçek kanaldır.



15.1 Analitik İşaret ve Hilbert Dönüşümü

Gerçek bir işaretin negatif frekanslarını silip pozitifleri iki katına çıkarırsak **analitik işareti** elde ederiz:

$$x_a(t) = x(t) + j\mathcal{H}\{x\}(t).$$

Bu işlem, gerçek bir kayıttan IQ üretmenin en pratik yoludur.

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs = 48000
5 t = np.arange(fs)/fs
6 x = np.cos(2*np.pi*1000*t)           # gercek isaret
  
```

```

7
8 xa = sig.hilbert(x)                                # analitik isaret (karmasik)
9 print(np.allclose(xa.real, x))                     # True -> gercek kısmi degismez
10 print(np.allclose(xa.imag[100:-100],
11                  np.sin(2*np.pi*1000*t)[100:-100], atol=1e-3)) # True
12
13 zarf = np.abs(xa)                                # anlik genlik ~1.0
14 faz = np.unwrap(np.angle(xa))                    # anlik faz
15 frekans = np.diff(faz)/(2*np.pi)*fs               # anlik frekans ~1000 Hz
16 print(np.median(frekans))                          # 1000.0

```

Dikkat

scipy.signal.hilbert FFT tabanlıdır: tüm diziyi bir kerede işler ve *dairesel* varsayım yüzünden uçlarda hata verir. Uzun akışlarda ya bloklara pencere+örtüşme uygula ya da FIR Hilbert filtresi (sig.remez(..., type='hilbert')) kullan.

15.2 IQ'nun Üç Yorumu

Bakış	Anlamı
Kartezyen	I ve Q : iki dik kanal. Modülatörde toplanır, alıcıda ayrılır.
Kutupsal	$ z $ genlik (AM bilgisi), $\arg z$ faz (FM/PM bilgisi).
Dönen vektör	$z[n]$ karmaşık düzlemde bir nokta; hızı frekans, yörüngesi takımıydız.

```

1 # Aynı IQ akisinden uc demodulasyon: AM, FM, PM
2 z = ...                                # complex64 bant temeli
3
4 am = np.abs(z)                          # zarf
5 fm = np.angle(z[1:]*np.conj(z[:-1]))    # faz farki
6 pm = np.unwrap(np.angle(z))             # faz

```

16. NCO, Mikser ve Frekans Kaydırma

Bir işareti frekansta kaydırmak, karmaşık bir üstelle çarpmaktır:

$$z'[n] = z[n] e^{j2\pi f_{\text{kay}} n / f_s}.$$

```

1 import numpy as np
2
3 def frekans_kaydir(z, kayma_hz, fs):
4     """IQ akisini kayma_hz kadar kaydırır (pozitif = yukari)."""
5     n = np.arange(len(z))
6     return z*np.exp(2j*np.pi*kayma_hz*n/fs).astype(z.dtype)
7
8 # Ornek: 2.4 MS/s IQ icinde +250 kHz'teki kanali merkeze getir
9 iq_merkez = frekans_kaydir(iq, -250e3, 2.4e6)

```

Dikkat

np.arange(len(z)) uzun akışlarda taşar ve *faz kesikliği* yaratır: bloklar arasında n 'i sıfırlarsan her blok sınırında faz sıçrar. Gerçek zamanlı hatta fazı biriktiren bir NCO nesnesi kullan.

```

1 class NCO:
2     """Blokler arası faz sürekliliğini koruyan sayısal osilatör."""
3     def __init__(self, fs, f0=0.0):
4         self.fs, self.f0, self.faz = fs, f0, 0.0
5
6     def uret(self, N):
7         adim = 2*np.pi*self.f0/self.fs
8         faz = self.faz + adim*np.arange(N)
9         self.faz = (self.faz + adim*N) % (2*np.pi)    # faz sarması
10        return np.exp(1j*faz)
11
12    def karistir(self, blok):
13        return blok*self.uret(len(blok))
14
15 nco = NCO(2.4e6, -250e3)
16 cikis = np.concatenate([nco.karistir(b) for b in bloklar])

```

Bilgi

Faz birikimini % (2π) ile sarmak sayısal hassasiyeti korur: aksi hâlde saatlerce çalışan bir alıcıda faz 10^{10} radyana ulaşır ve float64'ün çözünürlüğü içinde “titrek” bir osilatör elde edersin.

16.1 Gerçek İşareti Bant Temeline İndirmek

```

1 def gercekten_bant_temeline(x, fc, fs, bant, desim=1):
2     """Gerçek isarettteki fc merkezli kanalı karmasik bant temeline indirir."""
3     n = np.arange(len(x))
4     z = x*np.exp(-2j*np.pi*fc*n/fs)    # kanali 0 Hz'e kaydir
5     h = sig.firwin(129, bant/2, fs=fs)  # gorunutu bandi sinirla
6     z = sig.lfilter(h, 1.0, z)
7     return z[::desim] if desim > 1 else z

```

17. Desimasyon, İnterpolasyon ve Polifaz

SDR alıcısında örnekleme hızı 2,4 MS/s'ten 48 kS/s'e iner. Bu hız dönüşümü DSP'nin en sık yapılan işlemidir — ve en sık yanlış yapıları.

Altın Kural

Hızı M kat düşürmeden önce bandı $f_s/(2M)$ ile sınırla. Filtresiz desimasyon, bant dışı her şeyi bandın içine katlar (aliasing) ve geri döndürülemez.

Fonksiyon	Ne yapar?
<code>x[::M]</code>	Ham desimasyon — filtre yok , alias garantili.
<code>sig.decimate(x,M,ftype='fir')</code>	Anti-alias filtre + desimasyon. Basit ve güvenli.
<code>sig.resample_poly(x,L,M)</code>	Polifaz: L/M oranıyla rasyonel dönüşüm, en verimli yol.
<code>sig.resample(x,N)</code>	FFT tabanlı; periyodik varsayar, uçlarda hata verir.
<code>sig.upfirdn(h,x,L,M)</code>	Yukarı örnekle–filtre–indir; polifazın çekirdeği.

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs_in, fs_out = 2_400_000, 48_000
5 from math import gcd
6 g = gcd(fs_in, fs_out)
7 L, M = fs_out//g, fs_in//g
8 print(L, M)                                # 1 50    -> 50 kat desimasyon
9
10 ses = sig.resample_poly(bant_temeli.real, up=L, down=M)
11
12 # Çok büyük oranlarda kademeli in: 50 = 10 x 5 (daha kısa filtreler, daha hızlı)
13 ara = sig.resample_poly(bant_temeli.real, 1, 10)
14 ses = sig.resample_poly(ara, 1, 5)

```

İpucu

Tek adımda 50 kat indirmek yerine 10×5 kademeye bölmek toplam çarpma sayısını belirgin biçimde azaltır: ilk kademe yüksek hızda çalışır ama geçiş bandı geniş olduğu için kısa filtre yeter; son kademe dar filtre kullanır ama düşük hızda çalışır.

17.1 Polifaz Neden Hızlı?

Klasik yol: yukarı örnekle (araya sıfır koy), filtrele, indir. Bu, sonucu atılacak örnekler için de çarpma yapar. Polifaz ayrıştırma, filtreyi M alt filtreye bölerek *yalnızca tutulacak* örnekleri hesaplar — maliyet M kat düşer.

```

1 def polifaz_desimasyon(x, h, M):
2     """Sadece tutulacak örnekleri hesaplayan desimasyon (ogretici)."""
3     x = np.asarray(x)
4     N = len(x)//M*M
5     x = x[:N].reshape(-1, M).T                # M satirli polifaz kollari
6     h_pad = np.pad(h, (0, (-len(h)) % M))
7     hp = h_pad.reshape(-1, M).T
8     y = sum(np.convolve(x[m], hp[m], mode='full') for m in range(M))
9     return y

```

17.2 IQ Desimasyonu

Karmaşık işaretlerde I ve Q ayrı ayrı işlenir; resample_poly bunu otomatik yapar.

```

1 iq_dusuk = sig.resample_poly(iq, 1, 50)        # complex girdi destekli
2 # Elle: iq_dusuk = sig.resample_poly(iq.real,1,50) + 1j*sig.resample_poly(iq.imag,1,50)

```

18. IQ Kusurları: DC Ofset, Dengesizlik ve Düzeltme

Gerçek donanımda I ve Q kanalları asla mükemmel değildir. İki tipik kusur:

- **DC ofset:** LO sızıntısı, spektrumun tam ortasında sahte bir tepe yaratır.
- **IQ dengesizliği:** kanallar arasında kazanç (α) ve dikağı (ϕ) hatası; spektrumda *ayna görüntüsü* (image) oluşturur.

```

1 import numpy as np
2
3 def dc_kaldir(z):
4     return z - np.mean(z)                # blok blok da yapılabilir
5
6 def iq_dengele(z):
7     """Basit kazanç + dikacilik düzeltmesi (Gram-Schmidt yaklaşımı)."""
8     I, Q = z.real, z.imag
9     I = I - I.mean(); Q = Q - Q.mean()
10    alpha = np.sqrt(np.mean(I**2)/np.mean(Q**2))    # kazanç oranı
11    Q = Q*alpha
12    c = np.mean(I*Q)/np.mean(I**2)                # dikacilik hatası
13    Q = Q - c*I
14    return (I + 1j*Q).astype(z.dtype)
15
16 # Kusuru ölçmek: ayna bastırma oranı (IRR)
17 def irr_db(z, fs, f_ton):
18     N = len(z)
19     Z = np.fft.fftshift(np.abs(np.fft.fft(z)))
20     f = np.fft.fftshift(np.fft.fftfreq(N, 1/fs))
21     ana = Z[np.argmin(np.abs(f - f_ton))]
22     ayna = Z[np.argmin(np.abs(f + f_ton))]
23     return 20*np.log10(ana/ayna)

```

Örnek

Kusursuz bir alıcıda IRR sonsuzdur (ayna yok). Ucuz bir RTL-SDR'de tipik olarak 30–40 dB'dir; yukarıdaki düzeltme bunu 50–60 dB'ye çıkarabilir. Zayıf bir istasyonun yanında güçlü bir komşu varsa bu fark istasyonu duyabilmenle duyamamanın farkıdır.

18.1 Takımyıldız (Constellation) Çizimi

```

1 import matplotlib.pyplot as plt
2
3 def takimyildiz(z, baslik='', n=5000):
4     z = z[:n]
5     plt.figure(figsize=(5, 5))
6     plt.plot(z.real, z.imag, '.', ms=1.5, alpha=0.5, color='#2E6DA4')
7     plt.axhline(0, color='gray', lw=0.6); plt.axvline(0, color='gray', lw=0.6)
8     plt.gca().set_aspect('equal'); plt.grid(alpha=0.3)
9     plt.xlabel('I'); plt.ylabel('Q'); plt.title(baslik); plt.show()

```

Bilgi

Takımyıldız, bir modemın sağlık göstergesidir: noktalar dağınıksa gürültü/ISI, halka biçiminde dönüyorsa frekans ofseti, eğik elipsse IQ dengesizliği, bulanık ama merkezliyse zamanlama hatası vardır. Kısım 7'deki QPSK modeminde bu okumayı adım adım yapacağız.

KISIM 5

Modülasyon ve Demodülasyon

19. Genel Bakış: Modülasyon Neden Var?

Bilgi işaretimiz (ses, veri) alçak frekanslıdır; havada verimli yayılmaz. Modülasyon, bu bilgiyi yüksek frekanslı bir *taşıyıcının* bir özelliğine — genliğine, frekansına veya fazına — bindirir.

Tür	Değişen	Tipik kullanım
AM	genlik $A(t)$	AM radyo, havacılık telsizi, SSB.
FM	anlık frekans	FM radyo, telsiz, analog TV sesi.
PM	faz $\varphi(t)$	Sayısal modemlerin temeli (PSK).
ASK/OOK	genlik (ayrık)	RFID, kızılötesi, basit sensörler.
FSK	frekans (ayrık)	Bluetooth (GFSK), LoRa, modemler.
PSK	faz (ayrık)	Uydu, Wi-Fi (düşük hız), DVB.
QAM	genlik + faz	Wi-Fi, LTE/5G, kablo TV, DSL.

Hepsinin Ortak Denklemi

$$s(t) = \underbrace{A(t)}_{\text{AM}} \cos \left(2\pi f_c t + \underbrace{\varphi(t)}_{\text{PM}} + \underbrace{2\pi k_f \int m(\tau) d\tau}_{\text{FM}} \right)$$

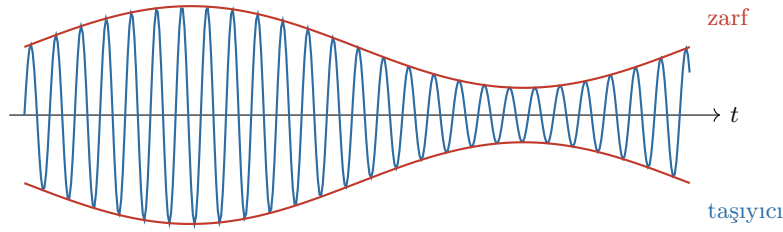
IQ dilinde hepsi tek bir karmaşık dizidir: $z[n] = A[n]e^{j\varphi[n]}$. Modülasyon türü, bilgiyi $|z|$ 'ye mi yoksa $\arg z$ 'ye mi koyduğundur.

20. AM: Genlik Modülasyonu

AM Denklemi

$$s(t) = A_c(1 + m x(t)) \cos(2\pi f_c t), \quad |x(t)| \leq 1$$

m **modülasyon indeksi**dir. $m > 1$ olursa zarf sıfırın altına iner, *aşırı modülasyon* olur ve zarf detektörü bozulur.



20.1 AM Modülatörü

```

1 import numpy as np
2 from scipy import signal as sig
3
4 fs = 200_000          # ornekleme
5 fc = 20_000           # taşıyıcı
6 fm = 500              # mesaj
7 m = 0.7               # modülasyon indeksi
8
9 t = np.arange(int(0.05*fs))/fs
10 mesaj = np.sin(2*np.pi*fm*t)
11 am = (1 + m*mesaj)*np.cos(2*np.pi*fc*t)
12
13 # Bant temeli karsiligi (IQ): faz sabit, genlik degisiyor
14 z_am = (1 + m*mesaj).astype(np.complex64)

```

AM Verimliliği

Toplam gücün taşıyıcıya giden kısmı bilgi taşımaz:

$$\eta = \frac{m^2/2}{1 + m^2/2} \Big|_{\text{sinüs mesaj}},$$

$m = 1$ için bile yalnızca %33. Bu yüzden taşıyıcıyı ve bir yan bandı silen **SSB** (tek yan bant) telsizde tercih edilir.

```

1 for m in [0.3, 0.5, 1.0]:
2     eta = (m**2/2)/(1 + m**2/2)
3     print(f"m={m:>4} verim = {100*eta:5.1f} %")
4 # m= 0.3 verim = 4.3 %
5 # m= 0.5 verim = 11.1 %
6 # m= 1.0 verim = 33.3 %

```

20.2 Zarf Detektörü ile Demodülasyon

En basit AM demodülasyonu: analitik işaretin mutlak değeri (ya da donanımda diyot + RC).

```

1 def am_zarf_demod(s, fs, ses_kesim=5000):
2     """Zarf detektörü: |analitik işaret| - DC."""
3     zarf = np.abs(sig.hilbert(s))
4     b = sig.firwin(129, ses_kesim, fs=fs)
5     zarf = sig.lfilter(b, 1.0, zarf)
6     return zarf - np.mean(zarf)          # taşıyıcı DC'sini at
7
8 ses = am_zarf_demod(am, fs)
9 print(np.corrcoef(ses[200:], mesaj[200:])[0, 1]) # ~0.999

```

```

1 # IQ akisından geliyorsa daha da kolay:
2 def am_demod_iq(z, fs, ses_kesim=5000):
3     zarf = np.abs(z)
4     b = sig.firwin(129, ses_kesim, fs=fs)
5     return sig.lfilter(b, 1.0, zarf - np.mean(zarf))

```

20.3 Senkron (Coherent) Demodülasyon

Zarf detektörü gürültüde zayıftır ve aşırı modülasyonda çalışmaz. Senkron detektör taşıyıcıyı yeniden üretip çarpır; 3 dB daha iyi SNR verir.

```

1 def am_senkron_demod(s, fs, fc, ses_kesim=5000):
2     """Tasiyiciyi yeniden uretip carpma (faz kilitli olmalı)."""
3     n = np.arange(len(s))
4     z = s*np.exp(-2j*np.pi*fc*n/fs)          # bant temeline indir
5     b = sig.firwin(129, ses_kesim, fs=fs)
6     z = sig.lfilter(b, 1.0, z)
7     # Faz hatasini gider: isaretin ortalama fazina gore dondur
8     z = z*np.exp(-1j*np.angle(np.mean(z)))
9     return z.real - np.mean(z.real)

```

Dikkat

Senkron demodülasyonda taşıyıcı fazı θ kadar kayarsa çıkış $\cos\theta$ ile çarpılır: $\theta = 90^\circ$ 'de ses **tamamen kaybolur**. Bu yüzden gerçek alıcılarda faz, bir PLL veya Costas döngüsüyle kilitlenir (bu kısmın sonunda).

21. FM: Frekans Modülasyonu

FM Denklemleri

$$s(t) = A_c \cos \left(2\pi f_c t + 2\pi k_f \int_0^t m(\tau) d\tau \right), \quad \Delta f = k_f \max |m|$$

Δf frekans sapmasıdır (FM radyoda 75 kHz). Modülasyon indeksi $\beta = \Delta f / f_m$ 'dir.

Carson Kuralı

FM işaretinin pratik bant genişliği:

$$B \approx 2(\Delta f + f_{m,\max}) = 2f_{m,\max}(\beta + 1).$$

FM radyo için $2(75 + 15) = 180$ kHz — kanal aralığının neden 200 kHz olduğunun cevabı.

21.1 Bessel: FM Spektrumu Neden Geniş?

FM'in yan bantları Bessel fonksiyonlarıyla verilir:

$$s(t) = A_c \sum_{n=-\infty}^{\infty} J_n(\beta) \cos(2\pi(f_c + nf_m)t).$$

```

1 from scipy.special import jv
2 import numpy as np
3
4 beta = 5.0
5 for n in range(8):
6     print(f"J_{n}({beta}) = {jv(n, beta):+.4f}")
7
8 # Carson kurali dogrulaması: gucun %98'ini kapsayan yan bant sayisi
9 guc = np.array([jv(n, beta)**2 for n in range(-40, 41)])
10 kum = np.cumsum(np.sort(guc)[::-1])/np.sum(guc)
11 print("gucun %98'i icin gereken bilesen:", np.searchsorted(kum, 0.98) + 1)

```

```

J_0(5.0) = -0.1776
J_1(5.0) = -0.3276
J_2(5.0) = +0.0466
J_3(5.0) = +0.3648
J_4(5.0) = +0.3912
J_5(5.0) = +0.2611
J_6(5.0) = +0.1310
J_7(5.0) = +0.0534

```

Bilgi

$J_0(\beta) = 0$ olduğu β değerlerinde (2,405, 5,520, ...) taşıyıcı tamamen kaybolur. Laboratuvarında frekans sapmasını ölçmenin klasik yolu budur: modülasyonu artırıp taşıyıcının söndüğü noktayı bulursun.

21.2 FM Modülatörü

```

1 import numpy as np
2
3 def fm_modulate(mesaj, fs, sapma_hz):
4     """Bant temeli FM: faz = 2*pi*sapma*kumulatif(mesaj)/fs."""
5     mesaj = mesaj/np.max(np.abs(mesaj)) # -1..1 normalize
6     faz = 2*np.pi*sapma_hz*np.cumsum(mesaj)/fs
7     return np.exp(1j*faz).astype(np.complex64)
8
9 fs = 240_000
10 t = np.arange(int(0.2*fs))/fs
11 ses = np.sin(2*np.pi*1000*t)
12 z_fm = fm_modulate(ses, fs, sapma_hz=75_000)

```

Dikkat

`np.cumsum` tam olarak integralin ayrık karşılığıdır ama uzun dizilerde float32 ile birikimli yuvarlama hatası yapar; faz için float64 kullan, sonucu complex64'e çevir.

21.3 FM Demodülasyonu: IQ Diskriminatör

FM demodülasyonu tek satırdır: ardışık örneklerin *faz farkı*, anlık frekansla orantılıdır.

Polar Diskriminatör

$$\hat{m}[n] = \arg(z[n] z^*[n-1]) \cdot \frac{f_s}{2\pi\Delta f}$$

Çarpım $z[n]z^*[n-1]$, iki fazör arasındaki açı farkını verir; `np.angle` bunu $[-\pi, \pi]$ aralığında güvenle döndürür (faz sarması sorunu yok).

```
1 import numpy as np
2
3 def fm_demod(z, fs=None, sapma_hz=None):
4     """IQ polar diskriminator. Ölçekleme isteğe bağlı."""
5     d = np.angle(z[1:]*np.conj(z[:-1]))
6     if fs and sapma_hz:
7         d = d*fs/(2*np.pi*sapma_hz)
8     return d
9
10 geri = fm_demod(z_fm, fs, 75_000)
11 print(np.corrcoef(geri[100:], ses[101:])[0, 1])    # ~0.9999
```

İpucu

`np.angle(z[1:]*np.conj(z[:-1]))` yerine `np.diff(np.unwrap(np.angle(z)))` de aynı sonucu verir ama daha yavaştır ve gürültüde unwrap hata yapabilir. Çarpım yöntemi hem hızlı hem sağlamdır — SDR alıcılarının standardıdır.

21.4 De-emphasis: FM'in Son Rötüşü

Vericide yüksek frekanslar yükseltilir (pre-emphasis), alıcıda aynı oranda bastırılır (de-emphasis); böylece FM'in yüksek frekanslarda artan gürültüsü azalır. Zaman sabiti Avrupa'da $50 \mu s$, ABD'de $75 \mu s$ 'tir.

```
1 from scipy import signal as sig
2
3 def deemphasis(x, fs, tau=50e-6):
4     """Tek kutuplu RC alçak gecirgen (bilinear dönüşüm ile)."""
5     # H(s) = 1/(1 + s*tau) -> sayısal karşılığı
6     b, a = sig.bilinear([1.0], [tau, 1.0], fs=fs)
7     return sig.lfilter(b, a, x)
```

Örnek

De-emphasis unutulursa FM radyo “tiz ve cızırtılı” duyulur; yanlış zaman sabiti kullanılırsa ses boğuk veya keskin olur. Bu, ev yapımı SDR alıcılarında en sık atlanan adımdır.

22. PM: Faz Modülasyonu**Formül**

$$s(t) = A_c \cos(2\pi f_c t + k_p m(t))$$

FM ile PM kardeşler: $m(t)$ 'nin integralini alıp PM uygularsan FM elde edersin. Bu yüzden pratikte tek bir IQ modülatör ikisini de üretir.

```

1 def pm_modulate(mesaj, k_p=1.0):
2     return np.exp(1j*k_p*mesaj).astype(np.complex64)
3
4 def pm_demod(z):
5     return np.unwrap(np.angle(z))
6
7 # FM ile PM ilişkisi
8 z_pm_of_integral = pm_modulate(np.cumsum(ses)*2*np.pi*75_000/fs)
9 print(np.allclose(z_pm_of_integral, fm_modulate(ses, fs, 75_000), atol=1e-5))

```

23. Sayısal Modülasyonlar: ASK, FSK, PSK, QAM

Sayısal modülasyonda bilgi *sürekli* değil, sonlu bir sembol kümesinden seçilir. Her sembol $\log_2 M$ bit taşır.

Şema	bit/sembol	E_b/N_0 @ 10^{-3}	Not
BPSK	1	6,8 dB	En dayanıklı; uzay haberleşmesi.
QPSK	2	6,8 dB	BPSK ile aynı BER, iki kat hız.
8-PSK	3	10,0 dB	Faz aralığı daralır, gürültüye duyarlı.
16-QAM	4	10,5 dB	Genlik + faz; doğrusal yükselteç ister.
64-QAM	6	14,8 dB	Yüksek SNR gerektirir (Wi-Fi, LTE).

23.1 BPSK ve QPSK

```

1 import numpy as np
2
3 rng = np.random.default_rng(0)
4 bitler = rng.integers(0, 2, 10_000)
5
6 # BPSK: 0 -> -1, 1 -> +1
7 bpsk = (2*bitler - 1).astype(np.complex64)
8
9 # QPSK (Gray kodlu): bit çiftleri -> 4 faz
10 b2 = bitler.reshape(-1, 2)
11 qpsk = ((1 - 2*b2[:, 0]) + 1j*(1 - 2*b2[:, 1]))/np.sqrt(2)
12
13 print(np.mean(np.abs(qpsk)**2))      # 1.0 -> birim ortalama güc

```

Gray Kodlama

Komşu takımyıldız noktaları yalnızca *tek bitte* farklı olacak şekilde eşlenirse, en sık görülen hata (komşuya kayma) tek bit hatası üretir. Bu, aynı SNR'de bit hata oranını yaklaşık yarıya indirir — bedelsiz bir kazanç.

23.2 FSK

```

1 def fsk_modulate(bitler, fs, sembol_hizi, ayirma_hz):
2     """Sürekli fazlı FSK (CPFSK): faz sicramasi yok."""

```

```

3     sps = int(fs/sembol_hizi)                                # sembol basına örnek
4     frek = np.where(bitler > 0, ayirma_hz/2, -ayirma_hz/2)
5     frek = np.repeat(frek, sps)
6     faz = 2*np.pi*np.cumsum(frek)/fs
7     return np.exp(1j*faz).astype(np.complex64)
8
9 def fsk_demod(z, fs, sembol_hizi, ayirma_hz):
10    d = np.angle(z[1:]*np.conj(z[:-1]))*fs/(2*np.pi)      # anlık frekans (Hz)
11    sps = int(fs/sembol_hizi)
12    d = d[:len(d)//sps*sps].reshape(-1, sps).mean(axis=1)
13    return (d > 0).astype(int)

```

Bilgi

Sürekli fazlı FSK (CPFSK) faz sıçraması içermediği için spektrumu dardır. Gauss filtresiyle yumuşatılmış hâli **GFSK**'dır ve Bluetooth ile birçok IoT telsizinin modülasyonudur.

23.3 QAM

```

1 def qam_takimyildiz(M):
2     """Kare M-QAM takimyildizi (birim ortalama guce normalize)."""
3     k = int(np.sqrt(M))
4     seviye = np.arange(-(k-1), k, 2)
5     I, Q = np.meshgrid(seviye, seviye)
6     c = (I + 1j*Q).ravel()
7     return c/np.sqrt(np.mean(np.abs(c)**2))
8
9 c16 = qam_takimyildiz(16)
10 print(len(c16), np.mean(np.abs(c16)**2))      # 16 1.0
11
12 def qam_karar(z, takim):
13     """En yakın komşu kararı (hard decision)."""
14     d = np.abs(z[:, None] - takim[None, :])
15     return np.argmin(d, axis=1)

```

Dikkat

QAM genlik bilgisi taşıdığı için vericinin güç yükselteci *doğrusal* olmalıdır; FM/FSK'de kullanılan verimli doyumlu (class-C) yükselteçler QAM'i bozar. Bu, 5G baz istasyonlarının neden pahalı doğrusal yükselteç kullandığının cevabıdır.

24. Darbe Şekillendirme ve Göz Diyagramı

Sembolleri kare darbelerle göndermek spektrumu sonsuz genişletir. Çözüm, darbeleri şekillendirmektir — ama şekil, komşu sembollere sızmamalıdır.

Nyquist ISI Kriteri

Bir darbe, sembol anlarında ($t = kT$, $k \neq 0$) sıfır oluyorsa **semboller arası girişim (ISI)** üretmez. *Raised cosine* ailesi bu koşulu sağlar; β (roll-off) bant genişliğini $(1 + \beta)/2T$ yapar.

```

1 import numpy as np
2
3 def rrc(beta, sps, span=10):

```

```

4     """Kök yükseltilmiş kosinus (root raised cosine) darbesi."""
5     N = span*sps
6     t = (np.arange(N + 1) - N/2)/sps
7     h = np.zeros_like(t)
8     for i, ti in enumerate(t):
9         if np.isclose(ti, 0.0):
10            h[i] = 1 - beta + 4*beta/np.pi
11        elif beta > 0 and np.isclose(abs(ti), 1/(4*beta)):
12            h[i] = (beta/np.sqrt(2))*((1 + 2/np.pi)*np.sin(np.pi/(4*beta))
13                                   + (1 - 2/np.pi)*np.cos(np.pi/(4*beta)))
14        else:
15            pay = (np.sin(np.pi*ti*(1 - beta))
16                  + 4*beta*ti*np.cos(np.pi*ti*(1 + beta)))
17            payda = np.pi*ti*(1 - (4*beta*ti)**2)
18            h[i] = pay/payda
19    return h/np.sqrt(np.sum(h**2))
20
21    sps, beta = 8, 0.35
22    h = rrc(beta, sps)
23
24    # Vericide sekillendirme: sembolleri sps kat yukari orneklendir, filtrele
25    semboller = qpsk
26    yukari = np.zeros(len(semboller)*sps, dtype=complex)
27    yukari[::sps] = semboller
28    tx = np.convolve(yukari, h, mode='same')

```

Bilgi

Kök yükseltilmiş kosinüs vericide *ve* alıcıda uygulanır; ikisinin çarpımı tam yükseltilmiş kosinüs olur ve ISI sıfırlanır. Buna *eşlenik filtre* (matched filter) denir ve aynı zamanda SNR'yi maksimize eder — iki kazanç bir arada.

24.1 Göz Diyagramı

```

1  import matplotlib.pyplot as plt
2
3  def goz_diyagrami(x, sps, n_iz=200, baslik=''):
4      """Ardisik 2 sembolluk parcalari ust uste cizer."""
5      x = np.real(x)
6      L = 2*sps
7      plt.figure(figsize=(6, 4))
8      for i in range(n_iz):
9          p = x[i*sps:i*sps + L]
10         if len(p) == L:
11             plt.plot(np.arange(L)/sps, p, color='#2E6DA4', alpha=0.15, lw=0.8)
12     plt.grid(alpha=0.3); plt.xlabel('Sembol'); plt.title(baslik); plt.show()
13
14  goz_diyagrami(np.convolve(tx, h, mode='same'), sps, baslik='RRC x RRC (ISI yok)')

```

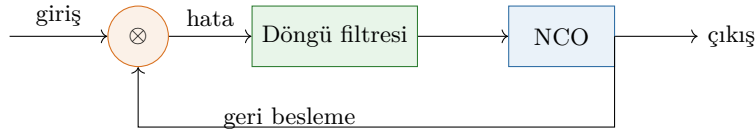
Örnek

Göz diyagramı okuma: *gözün açıklığı* gürültü payını, *dikey kalınlık* genlik bozulmasını, *yatay bulanıklık* zamanlama titreşimini (jitter) gösterir. Göz kapanmışsa ya ISI ya da SNR sorunu vardır.

25. Taşıyıcı ve Zamanlama Kurtarma

Alıcı, vericinin ne fazını ne de sembol saatini bilir. İki döngü bunu düzeltir: **taşıyıcı kurtarma** (faz/frekans) ve **zamanlama kurtarma** (örnekleme anı).

25.1 Faz Kilitlemeli Döngü (PLL)



```

1 import numpy as np
2
3 def pll(z, bant=0.01, zeta=0.707):
4     """İkinci mertebe PLL: taşıyıcı faz/frekans ofsetini düzeltir."""
5     # Doner Gardner/Rice katsayıları
6     Kp = 4*zeta*bant/(1 + 2*zeta*bant + bant**2)
7     Ki = 4*bant**2/(1 + 2*zeta*bant + bant**2)
8
9     faz, frek = 0.0, 0.0
10    cikis = np.empty_like(z)
11    for n in range(len(z)):
12        y = z[n]*np.exp(-1j*faz)
13        cikis[n] = y
14        hata = np.angle(y) # basit faz dedektörü
15        frek += Ki*hata
16        faz += frek + Kp*hata
17    return cikis

```

Dikkat

Bu döngü örnek başına Python yorumlayıcısında çalışır: 1 M örnek için saniyeler sürer. Gerçek kullanımda numba.njit ile derle (@njit eklemek 50–100 kat hızlandırır) veya bloklu kestirim kullan.

25.2 Costas Döngüsü: PSK için Taşıyıcı Kurtarma

BPSK/QPSK'da veri fazı $180^\circ/90^\circ$ atladığı için düz PLL kilitlenmez. Costas döngüsü, veri modülasyonunu kaldıran bir faz dedektörü kullanır.

```

1 def costas_qpsk(z, bant=0.005, zeta=0.707):
2     Kp = 4*zeta*bant/(1 + 2*zeta*bant + bant**2)
3     Ki = 4*bant**2/(1 + 2*zeta*bant + bant**2)
4     faz, frek = 0.0, 0.0
5     y = np.empty_like(z)
6     for n in range(len(z)):
7         v = z[n]*np.exp(-1j*faz)
8         y[n] = v
9         # QPSK faz dedektörü: karar yönlü (decision-directed)
10        hata = (np.sign(v.real)*v.imag - np.sign(v.imag)*v.real)
11        frek += Ki*hata
12        faz += frek + Kp*hata
13    return y

```

Bilgi

BPSK için faz dedektörü daha da basittir: $\text{hata} = v.\text{real} * v.\text{imag}$. Costas döngüsünün 180° (QPSK'da 90°) belirsizliği vardır: doğru kilitlenir ama hangi faza kilitlendiği bilinmez. Çözüm, *diferansiyel kodlama* ya da bilinen bir başlık (preamble) kullanmaktır.

25.3 Sembol Zamanlama Kurtarma: Gardner

```

1 def gardner_tzk(z, sps, kazanc=0.05):
2     """Gardner zamanlama hata dedektörü ile sembol ortasına kitleme."""
3     mu = 0.0                                # kesirli gecikme
4     i = sps
5     cikis = []
6     while i + sps < len(z):
7         idx = int(i)
8         simdi = z[idx]
9         orta = z[idx - sps//2]
10        onceki = z[idx - sps]
11        # Gardner hatasi: genlikten bagimsiz, tasiyici fazina duyarsiz
12        hata = (np.real(orta)*(np.real(simdi) - np.real(onceki))
13               + np.imag(orta)*(np.imag(simdi) - np.imag(onceki)))
14        mu += kazanc*hata
15        i += sps + mu
16        cikis.append(simdi)
17    return np.array(cikis)

```

İpucu

Gardner dedektörünün en güzel yanı, taşıyıcı fazı henüz kilitlenmemişken bile çalışmasıdır. Bu yüzden tipik alıcı sırası şudur: AGC → eşlenik filtre → zamanlama kurtarma → taşıyıcı kurtarma → karar. Kısım 7'deki QPSK modeminde bu zinciri baştan sona kuracağız.

KISIM 6

Uçtan Uca: Tam Bir FM Radyo Alıcısı

26. IQ Dosya Formatları ve RTL-SDR

SDR donanımları IQ örneklerini birkaç standart formatta verir. Doğru yorumlamak, alıcının ilk ve en kritik adımıdır.

Format	dtype	Nerede?
cu8	uint8	RTL-SDR ham çıktısı; merkez 127,5.
cs8	int8	HackRF.
cs16	int16	Airspy, SDRplay, birçok kayıt dosyası.
cf32	complex64	GNU Radio standardı; işlemede kullanılan tip.

```

1 import numpy as np
2
3 def cu8_oku(dosya, sayi=-1, offset=0):
4     """RTL-SDR ham (uint8, I/Q sıralı) dosyasını complex64'e çevirir."""
5     ham = np.fromfile(dosya, dtype=np.uint8,
6                       count=-1 if sayi < 0 else 2*sayi, offset=2*offset)
7     ham = ham.astype(np.float32)
8     ham = (ham - 127.5)/127.5 # -1..1 aralığına getir
9     return (ham[0::2] + 1j*ham[1::2]).astype(np.complex64)
10
11 def cf32_oku(dosya):
12     return np.fromfile(dosya, dtype=np.complex64)
13
14 def cs16_oku(dosya):
15     ham = np.fromfile(dosya, dtype=np.int16).astype(np.float32)/32768.0
16     return (ham[0::2] + 1j*ham[1::2]).astype(np.complex64)

```

Dikkat

En sık yapılan hata I/Q sırasını ters almaktır. Sonuç: spektrum aynaya düşer, FM sesi “tersine” çıkar. Şüphelenirsen `np.conj(iq)` deneyerek kontrol et — doğru olan, istasyonun beklenen frekansta görünmesidir.

26.1 Donanımdan Canlı Akış

```

1 from rtlsdr import RtlSdr
2 import numpy as np
3
4 sdr = RtlSdr()
5 sdr.sample_rate = 2.4e6          # 2.4 MS/s
6 sdr.center_freq = 100.1e6        # istasyon frekansı
7 sdr.freq_correction = 60         # ppm düzeltmesi
8 sdr.gain = 'auto'
9
10 iq = sdr.read_samples(256*1024) # complex128 doner
11 iq = iq.astype(np.complex64)
12 sdr.close()
13
14 # Surekli akis (async)
15 def kare_isle(ornekler, ctx):
16     ses = wbfm_al(ornekler.astype(np.complex64))
17     ...                               # ses cikisina yaz
18
19 # sdr.read_samples_async(kare_isle, 256*1024)

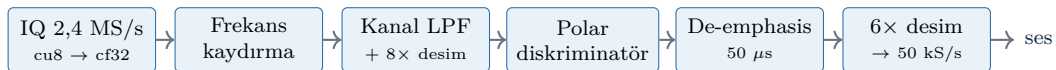
```

Bilgi

RTL-SDR'ın kristali tipik olarak 20–60 ppm kayar; 100 MHz'te bu 2–6 kHz'lik bir frekans hatası demektir. Geniş bant FM'de fark edilmez ama dar bant (NFM, SSB, sayısal) modlarda freq_correction ayarlanmazsa demodülasyon bozulur.

27. Tam WBFM Alıcı Hattı

Geniş bant FM (WBFM) alıcısı altı adımdır. Hepsi bu rehberde tek tek işlendi; şimdi birleştiriyoruz.



```

1 import numpy as np
2 from scipy import signal as sig
3 import soundfile as sf
4
5
6 class WBFMalıcı:
7     """RTL-SDR IQ akisinden ses ureten tam FM alıcısı."""
8
9     def __init__(self, fs_in=2_400_000, kanal_ofset=0.0,
10                  fs_ara=240_000, fs_ses=48_000, tau=50e-6):
11         self.fs_in = fs_in
12         self.ofset = kanal_ofset
13         self.dec1 = int(fs_in//fs_ara)          # 10
14         self.fs_ara = fs_in//self.dec1
15         self.dec2 = int(self.fs_ara//fs_ses)     # 5
16         self.fs_ses = self.fs_ara//self.dec2
17
18         # Kanal firtresi: WBFM icin +-100 kHz
19         self.h_kanal = sig.firwin(129, 100_000, fs=fs_in).astype(np.float32)
20         # Ses firtresi: 15 kHz'e kadar

```

```

21     self.h_ses = sig.firwin(129, 15_000, fs=self.fs_ara).astype(np.float32)
22     # De-emphasis
23     self.b_de, self.a_de = sig.bilinear([1.0], [tau, 1.0], fs=self.fs_ara)
24
25     # Bloklar arası durumlar
26     self.zi_kanal = np.zeros(len(self.h_kanal) - 1, dtype=np.complex64)
27     self.zi_ses = np.zeros(len(self.h_ses) - 1, dtype=np.float32)
28     self.zi_de = np.zeros(max(len(self.a_de), len(self.b_de)) - 1)
29     self.faz = 0.0
30     self.son_ornek = np.complex64(1.0)
31
32     def _kaydir(self, z):
33         n = np.arange(len(z))
34         adim = 2*np.pi*(-self.offset)/self.fs_in
35         lo = np.exp(1j*(self.faz + adim*n)).astype(np.complex64)
36         self.faz = (self.faz + adim*len(z)) % (2*np.pi)
37         return z*lo
38
39     def isle(self, iq):
40         z = iq.astype(np.complex64)
41         if self.offset:
42             z = self._kaydir(z)
43
44         # 1) Kanal filtresi + desimasyon
45         z, self.zi_kanal = sig.lfilter(self.h_kanal, 1.0, z, zi=self.zi_kanal)
46         z = z[:,self.dec1]
47
48         # 2) Polar diskriminasyon (blok sınırında faz sürekliliği için
49         # önceki bloğun son örneğini basa ekliyoruz)
50         z = np.concatenate((self.son_ornek, z))
51         self.son_ornek = z[-1]
52         ses = np.angle(z[1:]*np.conj(z[:-1])).astype(np.float32)
53
54         # 3) De-emphasis
55         ses, self.zi_de = sig.lfilter(self.b_de, self.a_de, ses, zi=self.zi_de)
56
57         # 4) Ses bandı filtresi + desimasyon
58         ses, self.zi_ses = sig.lfilter(self.h_ses, 1.0, ses, zi=self.zi_ses)
59         ses = ses[:,self.dec2]
60
61         # 5) Normalize
62         return (ses/(np.max(np.abs(ses)) + 1e-9)*0.7).astype(np.float32)
63
64     # Kullanım: kayıtlı bir dosyadan
65     alici = WBFMAlici(fs_in=2_400_000, kanal_offset=0.0)
66     iq = cu8_oku('kayit_100M1.iq')
67     ses = np.concatenate([alicis.isle(iq[i:i+262144])
68                           for i in range(0, len(iq), 262144)])
69     sf.write('radyo.wav', ses, alicis.fs_ses)
70     print(f"{len(ses)/alicis.fs_ses:.1f} saniye ses üretildi @ {alicis.fs_ses} Hz")

```

Dikkat

Blok blok işlerken üç şeyin durumu taşınmalıdır: (1) FIR/IIR filtre durumları (zi), (2) NCO fazı, (3) diskriminatörün bir önceki örneği. Üçü de yukarıdaki sınıfta saklanıyor. Biri unutulursa her blok sınırında duyulabilir bir “tık” oluşur.

27.1 Dar Bant FM (NFM) Farkı

Parametre	WBFM	NFM	Not
Sapma	75 kHz	2,5–5 kHz	NFM telsizlerde.
Kanal filtresi	100 kHz	6–8 kHz	Dar filtre → daha uzun FIR.
Ses bandı	15 kHz	3 kHz	Konuşma yeterli.
De-emphasis	50/75 μ s	750 μ s	Ya da hiç.

```

1 # NFM için aynı sınıf, farklı parametreler
2 nfm = WBFMAlici(fs_in=2_400_000, fs_ara=48_000, fs_ses=48_000, tau=750e-6)
3 nfm.h_kanal = sig.firwin(301, 8_000, fs=2_400_000).astype(np.float32)

```

28. Performans ve Gerçek Zamanlı Akış

28.1 Ne Kadar Hızlı Olmalı?

2,4 MS/s'lik bir akışı gerçek zamanlı işlemek için saniyede 2,4 milyon karmaşık örneği yetiştirmek gerekir. Doğru yazılmış NumPy kodu bunu rahatlıkla yapar; yanlış yazılmış Python döngüsü asla yapamaz.

Teknik	Kazanç
Erken desimasyon	En pahalı işlemleri en düşük hızda yap. Tek başına 10–50 kat.
complex64 kullan	Bellek yarı, FFT $\sim 1,7$ kat hızlı.
resample_poly	Polifaz, saf filtre+desimasyona göre M kat az çarpma.
sig.lfilter/sosfilt	C uygulaması; Python döngüsünden 100+ kat hızlı.
numba.njit	Kaçınılmaz örnek-bazlı döngüler (PLL, Viterbi) için 50–100 kat.
scipy.fft(workers=-1)	Çok çekirdekli FFT.
Önceden ayrılmış tampon	np.empty + out= ile blok başına ayırma yapma.

```

1 # Darbogazi olcmeden optimize etme
2 import cProfile, pstats
3
4 cProfile.run('alici.isle(iq[:262144])', 'prof.out')
5 pstats.Stats('prof.out').sort_stats('cumulative').print_stats(10)

```

28.2 Üretici–Tüketici ile Gerçek Zamanlı Alıcı

```

1 import queue, threading
2 import numpy as np, sounddevice as sd
3 from rtlsdr import RtlSdr
4
5 kuyruk = queue.Queue(maxsize=8)
6 alici = WBFMAlici()
7

```

```

8 def sdr_okuyucu():
9     sdr = RtlSdr()
10    sdr.sample_rate, sdr.center_freq, sdr.gain = 2.4e6, 100.1e6, 'auto'
11    try:
12        while calisiyor:
13            blok = sdr.read_samples(262144).astype(np.complex64)
14            try:
15                kuyruk.put_nowait(blok)
16            except queue.Full:
17                pass # gec kalan blogu at (drop)
18    finally:
19        sdr.close()
20
21 def ses_callback(outdata, frames, time, status):
22     global tampon
23     while len(tampon) < frames:
24         tampon = np.concatenate([tampon, alici.isle(kuyruk.get())])
25     outdata[:, 0] = tampon[:frames]
26     tampon = tampon[frames:]
27
28 calisiyor, tampon = True, np.zeros(0, dtype=np.float32)
29 threading.Thread(target=sdr_okuyucu, daemon=True).start()
30 with sd.OutputStream(samplerate=alici.fs_ses, channels=1,
31                     blocksize=2048, callback=ses_callback):
32     sd.sleep(60_000)
33 calisiyor = False

```

Dikkat

Kuyruk dolduğunda bloğu *atmak*, beklemekten iyidir: radyo canlı bir akıştır, gecikmiş veri işe yaramaz. Kuyruk sürekli doluyorsa işleme hattın gerçek zamandan yavaş demektir — desimasyonu öne al veya filtre uzunluğunu kısalt.

28.3 Nereye Bakmalı?

Kaynak	Ne için?
GNU Radio	Grafik akış tabanlı SDR çerçevesi; Python blokları yazılabilir.
SoapySDR	Donanımdan bağımsız sürücü katmanı (RTL, HackRF, USRP...).
PySDR (pysdr.org)	Python odaklı, ücretsiz ve çok iyi bir DSP/SDR kitabı.
scipy.signal belgeleri	Filtre tasarımı ve spektral analizin başvuru kaynağı.
Lyons, <i>Understanding DSP</i>	Sezgisel anlatımıyla klasik DSP kitabı.

KISIM 7

İleri Konular

29. Stereo FM Çözücü

FM yayınının bant temeli işareti (*multiplex*, MPX) birden çok bilgiyi frekansta ayırarak taşır:

Bant	İçerik
0–15 kHz	$L + R$ (mono uyumluluk için).
19 kHz	Pilot ton (stereo göstergesi ve faz referansı).
23–53 kHz	$L - R$, 38 kHz'te DSB-SC (bastırılmış taşıyıcı).
57 kHz	RDS/RBDS veri (1187,5 bps).
67/92 kHz	SCA (abonelik yayınları).

Stereo Matris

Pilotun *ikinci harmoniği* (38 kHz) ile $L - R$ bandını çarpıp alçak geçirirsek:

$$L = (L + R) + (L - R), \quad R = (L + R) - (L - R).$$

```

1 import numpy as np
2 from scipy import signal as sig
3
4 def fm_stereo_coz(mpx, fs_mpx, fs_ses=48_000):
5     """MPX isaretinden (diskriminator cikisi) L ve R kanallarini uretir."""
6     # 1) Mono: L+R (0-15 kHz)
7     h_mono = sig.firwin(257, 15_000, fs=fs_mpx)
8     lr_toplam = sig.lfilter(h_mono, 1.0, mpx)
9
10    # 2) Pilot: 19 kHz dar bant geciren
11    sos_pilot = sig.butter(4, [18_800, 19_200], btype='bandpass',
12                          fs=fs_mpx, output='sos')
13    pilot = sig.sosfiltfilt(sos_pilot, mpx)
14
15    # 3) Pilotun 2. harmonigini uret: analitik isaretin karesi
16    pilot_a = sig.hilbert(pilot)
17    tasiyici38 = np.real(pilot_a**2/(np.abs(pilot_a)**2 + 1e-12)) # cos(2*w*t)
18

```

```

19 # 4) L-R bandini asagi cek ve alcak gecir
20 sos_lr = sig.butter(6, [23_000, 53_000], btype='bandpass',
21                    fs=fs_mpx, output='sos')
22 lr_fark = sig.sosfiltfilt(sos_lr, mpx)*tasiyici38*2.0
23 lr_fark = sig.lfilter(h_mono, 1.0, lr_fark)
24
25 # 5) Matris ve ses hizina indirme
26 L = lr_toplam + lr_fark
27 R = lr_toplam - lr_fark
28 M = int(fs_mpx//fs_ses)
29 return sig.resample_poly(L, 1, M), sig.resample_poly(R, 1, M)
30
31
32 def stereo_var_mi(mpx, fs_mpx, esik_db=-30):
33     """19 kHz pilotun varligina bakarak stereo olup olmadigini soyler."""
34     N = min(len(mpx), 1 << 16)
35     X = np.abs(np.fft.rfft(mpx[:N]*np.hanning(N)))
36     f = np.fft.rfftfreq(N, 1/fs_mpx)
37     k19 = np.argmin(np.abs(f - 19_000))
38     seviye = 20*np.log10(X[k19]/np.max(X) + 1e-12)
39     return seviye > esik_db, seviye

```

Dikkat

Stereo çözmek için MPX işaretini **en az 100–120 kS/s** hızda tutmalıyız: 53 kHz'lik bileşeni koruman gerekir. WBFM alıcısında ara hızı 240 kS/s'te bırakıp stereo matristen *sonra* 48 kS/s'e in.

Bilgi

Pilotun karesini almak yerine faz kilitli bir PLL ile 38 kHz üretmek gürültüde daha sağlamdır: pilot zayıfladığında karesi alma yöntemi gürültüyü de kareler ve kanal ayrımı (separation) düşer.

30. Otomatik Kazanç Kontrolü (AGC)

Alınan işaret seviyesi saniyeler içinde 40 dB değişebilir. AGC, çıkışı sabit bir seviyeye getirir; demodülatörlerin ve karar devrelerinin doğru çalışması için şarttır.

Logaritmik AGC

$$g[n+1] = g[n] \cdot \exp\left(\mu(\log A_{\text{hedef}} - \log |g[n]x[n]|)\right)$$

Logaritmik alanda çalışmak, kazancın giriş seviyesinden *bağımsız* bir hızda yakınsamasını sağlar.

```

1 import numpy as np
2
3 def agc(x, hedef=1.0, mu=1e-3, g0=1.0):
4     """Ornek bazli logaritmik AGC (numba ile derlemeye uygun)."""
5     y = np.empty_like(x)
6     g = g0
7     for n in range(len(x)):
8         y[n] = x[n]*g
9         seviye = abs(y[n]) + 1e-12
10        g *= np.exp(mu*(np.log(hedef) - np.log(seviye)))

```

```

11     return y
12
13 def agc_blok(x, hedef=1.0, blok=1024, alpha=0.1):
14     """Cok daha hizli blok-bazli AGC: NumPy ile vektorlestirilmis."""
15     N = len(x)//blok*blok
16     p = np.sqrt(np.mean(np.abs(x[:N].reshape(-1, blok))**2, axis=1)) + 1e-12
17     g = hedef/p
18     # Kazanci yumusat (ani sicramalari onle)
19     g_yum = sig.lfilter([alpha], [1, -(1 - alpha)], g)
20     return (x[:N].reshape(-1, blok)*g_yum[:, None]).ravel()

```

İpucu

AGC'nin zaman sabiti kritiktir: çok hızlıysa modülasyonun kendisini (AM bilgisini) siler, çok yavaşsa güçlü darbelerde kırpar. Kural: AGC bandı, en düşük bilgi frekansının en az 10 kat altında olsun. AM demodülasyonunda AGC'yi *zarf detektöründen sonra* değil, önce ve yavaş uygula.

31. Uçtan Uca Sayısal Modem: QPSK

Şimdi bütün parçaları birleştirip çalışan bir modem kuralım: veri → RRC → kanal (gürültü + frekans/faz ofseti) → AGC → eşlenik filtre → zamanlama → taşıyıcı → karar → BER.

31.1 Verici

```

1 import numpy as np
2 from scipy import signal as sig
3
4 rng = np.random.default_rng(2)
5 sps, beta, span = 8, 0.35, 10
6 h = rrc(beta, sps, span) # Kisim 5'teki fonksiyon
7
8 def qpsk_verici(bit_sayisi=20_000):
9     bitler = rng.integers(0, 2, bit_sayisi)
10    b2 = bitler.reshape(-1, 2)
11    sym = ((1 - 2*b2[:, 0]) + 1j*(1 - 2*b2[:, 1]))/np.sqrt(2)
12    yukari = np.zeros(len(sym)*sps, dtype=complex)
13    yukari[::sps] = sym
14    tx = np.convolve(yukari, h, mode='full')
15    return bitler, sym, tx.astype(np.complex64)

```

31.2 Kanal Modeli

```

1 def kanal(tx, snr_db=15, frek_offset=0.0, faz_offset=0.0, fs=1.0, gecikme=0.0):
2     """AWGN + tasiyici ofseti + kesirli gecikme."""
3     n = np.arange(len(tx))
4     z = tx*np.exp(2j*np.pi*frek_offset*n/fs + 1j*faz_offset)
5
6     if gecikme: # kesirli gecikme (Lagrange)
7         k = np.arange(-4, 5)
8         h_g = np.sinc(k - gecikme)*np.hanning(len(k))
9         z = np.convolve(z, h_g, mode='same')
10
11    guc = np.mean(np.abs(z)**2)

```

```

12     n0 = guc/(10**(snr_db/10))
13     gurultu = np.sqrt(n0/2)*(rng.normal(size=len(z)) + 1j*rng.normal(size=len(z)))
14     return (z + gurultu).astype(np.complex64)

```

31.3 Alıcı

```

1 def qpsk_alici(rx, sps, h, kaba_frekans_duzelt=True):
2     # 1) AGC
3     rx = rx/np.sqrt(np.mean(np.abs(rx)**2))
4
5     # 2) Kaba frekans ofseti kestirimi: 4. kuvvet yontemi
6     if kaba_frekans_duzelt:
7         X = np.fft.fft(rx**4)
8         k = np.argmax(np.abs(X))
9         f4 = np.fft.fftfreq(len(rx))[k]
10        rx = rx*np.exp(-2j*np.pi*(f4/4)*np.arange(len(rx)))
11
12    # 3) Eslenik filtre (matched filter)
13    y = np.convolve(rx, h, mode='full')
14
15    # 4) Zamanlama kurtarma
16    sym = gardner_tzk(y, sps) # Kisim 5'teki fonksiyon
17
18    # 5) Ince tasiyici kurtarma (Costas)
19    sym = costas_qpsk(sym, bant=0.01)
20
21    # 6) Karar
22    bit0 = (sym.real < 0).astype(int)
23    bit1 = (sym.imag < 0).astype(int)
24    return np.column_stack([bit0, bit1]).ravel(), sym

```

31.4 BER Ölçümü

```

1 import numpy as np
2 from scipy.special import erfc
3
4 def ber_teorik_qpsk(ebn0_db):
5     ebn0 = 10**(np.asarray(ebn0_db)/10)
6     return 0.5*erfc(np.sqrt(ebn0))
7
8 # Ideal kanalda (senkronizasyon mukemmel) simulasyon
9 def ber_sim(ebn0_db, n_bit=2_000_000):
10    ebn0 = 10**(ebn0_db/10)
11    b = rng.integers(0, 2, n_bit).reshape(-1, 2)
12    s = ((1 - 2*b[:, 0]) + 1j*(1 - 2*b[:, 1]))/np.sqrt(2)
13    n0 = 1/(2*ebn0) # QPSK: Es = 2*Eb = 1
14    r = s + np.sqrt(n0/2)*(rng.normal(size=len(s)) + 1j*rng.normal(size=len(s)))
15    bh = np.column_stack([(r.real < 0).astype(int), (r.imag < 0).astype(int)])
16    return np.mean(bh != b)
17
18 for db in [0, 2, 4, 6, 8, 10]:
19     print(f"Eb/NO = {db:>2} dB    sim = {ber_sim(db, 400_000):.2e}    "
20           f"teorik = {ber_teorik_qpsk(db):.2e}")

```

Eb/NO = 0 dB	sim = 7.87e-02	teorik = 7.86e-02
Eb/NO = 2 dB	sim = 3.76e-02	teorik = 3.75e-02

Eb/NO = 4 dB	sim = 1.24e-02	teorik = 1.25e-02
Eb/NO = 6 dB	sim = 2.42e-03	teorik = 2.39e-03
Eb/NO = 8 dB	sim = 1.90e-04	teorik = 1.91e-04
Eb/NO = 10 dB	sim = 5.00e-06	teorik = 3.87e-06

Dikkat

Son satırdaki fark simülasyon hatasıdır, hata değil: $400\,000$ bitte $3,87 \times 10^{-6}$ oramı ortalama $1,5$ hata demektir. Düşük BER ölçmek için en az $100/\text{BER}$ bit gerekir — 10^{-6} için yüz milyon bit.

Bilgi

Simülasyonun teoriyle örtüşmesi, modemın doğru kurulduğunun en güçlü kanıtıdır. Gerçek alıcıda eğri sağa kayar; aradaki farka *uygulama kaybı* (implementation loss) denir ve tipik olarak 0,5–2 dB'dir. Kaybın kaynağı senkronizasyon hataları, filtre kusurları ve kuantalamadır.

32. OFDM: Çok Taşıyıcılı Modülasyon

OFDM, veriyi yüzlerce dar bantlı alt taşıyıcıya böler. Her alt taşıyıcının bandı çok dar olduğu için kanal onları *düz* görür — çok yollu yayılıma karşı en etkili çözüm. Wi-Fi, LTE, 5G, DVB-T ve DAB bunu kullanır.

OFDM'in Sırrı

Alt taşıyıcı üretimi ve toplanması tam olarak bir **IFFT**'dir; alıcıda ayrıştırma bir **FFT**'dir. Böylece N modülatör yerine tek bir FFT yeter.

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{j2\pi kn/N}$$

```

1 import numpy as np
2
3 N_alt, N_cp = 64, 16                                # alt taşıyıcı ve döngüsel önek uzunluğu
4
5 def ofdm_verici(semboller, N=N_alt, cp=N_cp):
6     """semboller: (n_sembol, N) şekilli QAM dizisi."""
7     zaman = np.fft.ifft(semboller, axis=1)*np.sqrt(N)
8     with_cp = np.hstack([zaman[:, -cp:], zaman])      # döngüsel önek
9     return with_cp.ravel()
10
11 def ofdm_alici(rx, N=N_alt, cp=N_cp):
12     kareler = rx.reshape(-1, N + cp)[: , cp:]        # CP at
13     return np.fft.fft(kareler, axis=1)/np.sqrt(N)
14
15 # Kanal denkleştirme: pilot alt taşıyıcılardan H kestirimi
16 def kanal_kestir(alinan_pilot, gönderilen_pilot):
17     return alinan_pilot/gönderilen_pilot              # zero-forcing
18
19 rng = np.random.default_rng(0)
20 c16 = qam_takimyildiz(16)
21 sym = c16[rng.integers(0, 16, (100, N_alt))]
22 tx = ofdm_verici(sym)
23 rx_sym = ofdm_alici(tx)
24 print(np.max(np.abs(rx_sym - sym)))                  # ~1e-15 -> kayıpsız geri dönüş

```

Dikkat

OFDM'in iki büyük zaafı vardır: (1) yüksek **tepe/ortalama güç oranı** (PAPR) — yükseltici doğrusallığı zorlar; (2) frekans ofsetine aşırı duyarlılık — alt taşıyıcılar arası dikliği bozar (ICI). Bu yüzden OFDM alıcıları hassas frekans senkronizasyonu (Schmidl–Cox) kullanır.

```
1 def papr_db(x):
2     return 10*np.log10(np.max(np.abs(x)**2)/np.mean(np.abs(x)**2))
3
4 print(f"OFDM PAPR: {papr_db(tx):.1f} dB")      # tipik 8-12 dB
5 print(f"QPSK PAPR: {papr_db(qpsk):.1f} dB")    # ~0 dB (sabit zarf)
```

32.1 Döngüsel Önek (Cyclic Prefix) Neden Var?

Çok yollu yayılım, sembolleri birbirine karıştırır. Sembolün sonundan bir parçayı başına kopyalarsak, gecikmiş yansımalar hâlâ *aynı* sembolün içine düşer ve doğrusal konvolüsyon *dairese*l konvolüsyona dönüşür — bu da frekans alanında basit bir çarpma demektir. Kanalı düzeltmek tek bir bölmeye indirgenir.

33. Kanal Kodlaması: Konvolüsyonel Kod ve Viterbi

Gürültü bitleri bozar; kanal kodlaması bunları *düzeltilir*. Konvolüsyonel kod, her giriş bitini kaydırmalı bir yazmaçtan geçirerek fazla (redundant) bit üretir.

```
1 import numpy as np
2
3 # K=3, oran 1/2, ureticiler: 7 (111) ve 5 (101) -- klasik NASA kodu
4 G = [0o7, 0o5]
5 K = 3
6
7 def konv_kodla(bitler, G=G, K=K):
8     durum = 0
9     cikis = []
10    for b in np.append(bitler, np.zeros(K - 1, dtype=int)): # kuyruk bitleri
11        yazmac = (b << (K - 1)) | durum
12        for g in G:
13            cikis.append(bin(yazmac & g).count('1') % 2)
14        durum = yazmac >> 1
15    return np.array(cikis, dtype=int)
```

```
1 def viterbi_coz(alinan, G=G, K=K):
2     """Hard-decision Viterbi cozucu."""
3     n_durum = 1 << (K - 1)
4     n_cikis = len(G)
5     inf = 10**9
6
7     # Gecis tablosu: (durum, giris) -> (yeni_durum, cikis bitleri)
8     gecis = {}
9     for s in range(n_durum):
10        for b in (0, 1):
11            yazmac = (b << (K - 1)) | s
12            out = tuple(bin(yazmac & g).count('1') % 2 for g in G)
13            gecis[(s, b)] = (yazmac >> 1, out)
```

```

14
15 T = len(alinan)//n_cikis
16 metrik = np.full(n_durum, inf); metrik[0] = 0
17 yol = np.zeros((T, n_durum), dtype=np.int8)
18
19 for t in range(T):
20     r = alinan[t*n_cikis:(t + 1)*n_cikis]
21     yeni = np.full(n_durum, inf)
22     yeni_yol = np.zeros(n_durum, dtype=np.int8)
23     for s in range(n_durum):
24         if metrik[s] >= inf:
25             continue
26         for b in (0, 1):
27             ns, out = gecis[(s, b)]
28             d = metrik[s] + int(np.sum(np.abs(np.array(out) - r)))
29             if d < yeni[ns]:
30                 yeni[ns], yeni_yol[ns] = d, (s << 1) | b
31     metrik, yol[t] = yeni, yeni_yol
32
33 # Geri izleme
34 s = int(np.argmin(metrik))
35 bitler = []
36 for t in range(T - 1, -1, -1):
37     v = yol[t][s]
38     bitler.append(v & 1)
39     s = v >> 1
40 return np.array(bitler[::-1][:T - (K - 1)])

```

```

1 # Dogrulama: rastgele bit hatalari ekleyip duzeltilebiliyor mu?
2 rng = np.random.default_rng(0)
3 veri = rng.integers(0, 2, 200)
4 kod = konv_kodla(veri)
5
6 bozuk = kod.copy()
7 hata_idx = rng.choice(len(kod), size=8, replace=False) # 8 bit boz
8 bozuk[hata_idx] ^= 1
9
10 coz = viterbi_coz(bozuk)
11 print("hatali bit sayisi:", np.sum(coz != veri)) # 0 -> tamamı düzeltildi

```

Bilgi

Bu kodun *serbest mesafesi* $d_{\text{free}} = 5$ 'tir: en fazla $\lfloor (5 - 1)/2 \rfloor = 2$ ardışık hatayı garantili düzeltir, dağınık hataları ise çok daha fazlasını. Yumuşak karar (soft-decision) Viterbi, bit yerine gerçek genlik değerlerini kullanır ve ~ 2 dB ek kazanç sağlar — metrikte $\text{np.abs}(\text{out} - \text{r})$ yerine Öklid uzaklığı kullanmak yeter.

KISIM 8

Projeler

34. Proje 1 — DTMF (Telefon Tuşu) Çözücü

Telefon tuşları, biri satır biri sütun olmak üzere *iki* tonun toplamıyla kodlanır. Tek bir frekansı ölçmek için tam FFT gereksizdir; **Goertzel** algoritması tek bin'i $O(N)$ maliyetle hesaplar.

Hz	1209	1336	1477	1633
697	1	2	3	A
770	4	5	6	B
852	7	8	9	C
941	*	0	#	D

```

1 import numpy as np
2
3 def goertzel(x, f_hedef, fs):
4     """Tek frekans bileşeninin gücü: O(N), FFT'siz."""
5     N = len(x)
6     k = int(0.5 + N*f_hedef/fs)
7     w = 2*np.pi*k/N
8     katsayi = 2*np.cos(w)
9     s1 = s2 = 0.0
10    for ornek in x:
11        s0 = ornek + katsayi*s1 - s2
12        s2, s1 = s1, s0
13    return s1*s1 + s2*s2 - katsayi*s1*s2    # güc
14
15
16 SATIR = [697, 770, 852, 941]
17 SUTUN = [1209, 1336, 1477, 1633]
18 TUSLAR = ['123A', '456B', '789C', '*0#D']
19
20 def dtmf_coz(x, fs, esik=1e-3):
21     gs = [goertzel(x, f, fs) for f in SATIR]
22     gc = [goertzel(x, f, fs) for f in SUTUN]
23     i, j = int(np.argmax(gs)), int(np.argmax(gc))
24     toplam = sum(gs) + sum(gc)
25     if toplam < esik or gs[i]/(sum(gs) + 1e-12) < 0.5:
26         return None    # tus yok / belirsiz

```

```

27     return TUSLAR[i][j]
28
29
30 # Test: "5" tusu uret ve coz
31 fs = 8000
32 t = np.arange(int(0.1*fs))/fs
33 tus5 = np.sin(2*np.pi*770*t) + np.sin(2*np.pi*1336*t)
34 print(dtmf_coz(tus5, fs))          # 5

```

```

1 def dtmf_dizi_coz(x, fs, pencere=0.04, adim=0.02):
2     """Bir ses kaydındaki tus dizisini cozer (ardisik tekrarları birlestirir)."""
3     W, H = int(pencere*fs), int(adim*fs)
4     sonuc, onceki = [], None
5     for i in range(0, len(x) - W, H):
6         t = dtmf_coz(x[i:i+W], fs)
7         if t != onceki and t is not None:
8             sonuc.append(t)
9             onceki = t
10    return ''.join(sonuc)

```

İpucu

Goertzel'i vektörleştirmek istersen `sig.lfilter` ile ikinci mertebeden özyinelemeli filtre olarak yazabilirsin: `b=[1,-exp(-1j*w)]`, `a=[1,-2*cos(w),1]`. Python döngüsünden 100 kat hızlanır.

35. Proje 2 — Ses Üzerinden Veri İletimi (FSK Modem)

Hoparlörden veri gönderip mikrofonla almak: modemlerin ilk günlerinden kalma, hâlâ öğretici bir proje.

```

1 import numpy as np
2 from scipy import signal as sig
3
4 FS = 44100
5 BAUD = 100                                # sembol/saniye
6 F0, F1 = 1200.0, 2200.0                  # Bell 202 benzeri
7 SPS = int(FS/BAUD)
8 ONEK = [1, 0]*16 + [1, 1, 0, 0]         # senkron oneki (preamble)
9
10
11 def bayt_to_bit(veri: bytes):
12     return np.unpackbits(np.frombuffer(veri, dtype=np.uint8))
13
14 def bit_to_bayt(bitler):
15     n = len(bitler)//8*8
16     return np.packbits(bitler[:n].astype(np.uint8)).tobytes()
17
18 def fsk_gonder(veri: bytes):
19     bitler = np.concatenate([ONEK, bayt_to_bit(veri)])
20     frek = np.where(bitler > 0, F1, F0)
21     frek = np.repeat(frek, SPS)
22     faz = 2*np.pi*np.cumsum(frek)/FS      # surekli faz (CPFSK)
23     x = 0.5*np.sin(faz)
24     # Yumusak baslangic/bitis: tik seslerini onle
25     ramp = np.hanning(2*SPS)

```

```

26     x[:SPS] *= ramp[:SPS]; x[-SPS:] *= ramp[SPS:]
27     return x.astype(np.float32)
28
29
30 def fsk_al(x):
31     """Uyumlu (non-coherent) FSK alıcı: iki bant geciren + zarf karsilastirma."""
32     def zarf(f):
33         sos = sig.butter(4, [f - 300, f + 300], btype='bandpass',
34                          fs=FS, output='sos')
35         return np.abs(sig.hilbert(sig.sosfiltfilt(sos, x)))
36
37     e0, e1 = zarf(F0), zarf(F1)
38     karar = e1 - e0
39
40     # Sembol zamanlamasi: karar isaretinin sifir gecislerinden faz kestir
41     gecis = np.flatnonzero(np.diff(np.sign(karar)))
42     ofset = int(np.median(gecis % SPS)) if len(gecis) else SPS//2
43     orta = np.arange(ofset + SPS//2, len(karar), SPS)
44     bitler = (karar[orta] > 0).astype(int)
45
46     # Oneki bul ve at
47     o = np.array(ONEK)
48     for i in range(len(bitler) - len(o)):
49         if np.array_equal(bitler[i:i+len(o)], o):
50             return bit_to_bayt(bitler[i+len(o):])
51     return b''
52
53
54 # Dosyaya yaz / oku ile test
55 x = fsk_gonder(b'MERHABA DSP')
56 print(fsk_al(x))                                # b'MERHABA DSP'

```

Dikkat

Gerçek akustik kanalda oda yankısı (çok yolluluk) ve hoparlör-mikrofon frekans yanıtı işareti bozar. Baud hızını düşürmek (100–300), tonları birbirinden uzak seçmek ve hata düzeltme kodu (Kısım 7) eklemek güvenilirliği belirgin biçimde artırır.

36. Proje 3 — Gerçek Zamanlı Spektrum Analizörü

```

1  import numpy as np, sounddevice as sd
2  import matplotlib.pyplot as plt
3  from matplotlib.animation import FuncAnimation
4  from scipy import signal as sig
5
6  FS, N = 44100, 2048
7  pencere = sig.get_window('hann', N)
8  tampon = np.zeros(N, dtype=np.float32)
9
10 def ses_callback(indata, frames, time, status):
11     global tampon
12     yeni = indata[:, 0]
13     tampon = np.concatenate([tampon, yeni])[-N:]
14
15 fig, ax = plt.subplots(figsize=(9, 4))

```

```

16 f = np.fft.rfftfreq(N, 1/FS)
17 çizgi, = ax.semilogx(f[1:], np.zeros(len(f) - 1), lw=1.2, color='#2E6DA4')
18 ax.set_xlim(20, FS/2); ax.set_ylim(-100, 0); ax.grid(alpha=0.3, which='both')
19 ax.set_xlabel('Frekans (Hz)'); ax.set_ylabel('dBFS')
20
21 def guncelle(_):
22     X = np.fft.rfft(tampon*pencere)
23     mag = 20*np.log10(np.abs(X)/np.sum(pencere)*2 + 1e-12)
24     çizgi.set_ydata(mag[1:])
25     return çizgi,
26
27 with sd.InputStream(samplerate=FS, channels=1, blocksize=512,
28                     callback=ses_callback):
29     ani = FuncAnimation(fig, guncelle, interval=30, blit=True, cache_frame_data=False)
30     plt.show()

```

İpucu

Ekranda titremeyi azaltmak için spektrumu üstel ortalamayla yumuşat: $ort = 0.8*ort + 0.2*mag$. Tepe tutucu (peak hold) eklemek de kolaydır: $tepe = np.maximum(tepe*0.995, mag)$.

37. Proje 4 — Enstrüman Akort Cihazı

Perde (pitch) kestirimi için FFT tepesi yetersizdir: en güçlü tepe çoğu zaman temel frekans değil bir harmoniktir. **Otokorelasyon** ve onun geliştirilmiş hâli **YIN**, periyodu doğrudan zaman alanında bulur.

```

1 import numpy as np
2
3 NOTALAR = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B']
4
5 def perde_otokorelasyon(x, fs, fmin=50, fmax=1000):
6     """FFT tabanlı hızlı otokorelasyonla temel frekansi bulur."""
7     x = x - np.mean(x)
8     N = len(x)
9     X = np.fft.rfft(x, 2*N)
10    r = np.fft.irfft(X*np.conj(X))[:N] # otokorelasyon
11    r /= (r[0] + 1e-12)
12
13    lag_min, lag_max = int(fs/fmax), int(fs/fmin)
14    seg = r[lag_min:lag_max]
15    lag = lag_min + int(np.argmax(seg))
16
17    # Parabolik enterpolasyon: alt-ornek doğruluk
18    if 0 < lag < N - 1:
19        a, b, c = r[lag-1], r[lag], r[lag+1]
20        lag = lag + 0.5*(a - c)/(a - 2*b + c + 1e-12)
21    return fs/lag
22
23
24 def nota_bul(f):
25     """Frekanstan nota adı ve sent (cent) sapması."""
26     n = 12*np.log2(f/440.0) + 69 # MIDI nota numarası
27     yakin = int(round(n))
28     sent = (n - yakin)*100

```

```
29     return f"{NOTALAR[yakin % 12]}{yakin//12 - 1}", sent
30
31
32 for f in [82.41, 110.0, 146.83, 196.0, 246.94, 329.63]:    # gitar telleri
33     ad, sent = nota_bul(f)
34     print(f"{f:7.2f} Hz -> {ad:>3} ({sent:+.1f} sent)")
```

```
82.41 Hz -> E2 (+0.1 sent)
110.00 Hz -> A2 (+0.0 sent)
146.83 Hz -> D3 (-0.0 sent)
196.00 Hz -> G3 (+0.0 sent)
246.94 Hz -> B3 (-0.0 sent)
329.63 Hz -> E4 (+0.0 sent)
```

Bilgi

Bir *sent*, yarım tonun yüzde biridir. İnsan kulağı yaklaşık 5–10 sentlik farkı ayırt eder; iyi bir akort cihazı ± 1 sent çözünürlükte olmalıdır. Parabolik enterpolasyon olmadan, 44,1 kHz’te 440 Hz için çözünürlük ancak ~ 4 senttir — bu yüzden enterpolasyon şarttır.

KISIM 9

Matematiksel Temeller

38. Karmaşık Üsteller ve Diklik

DFT'nin işe yaramasının nedeni tek bir özelliktir: farklı frekanslardaki karmaşık üsteller birbirine **diktir** (ortogonal).

Diklik İlişkisi

$$\sum_{n=0}^{N-1} e^{j2\pi kn/N} e^{-j2\pi mn/N} = \begin{cases} N, & k \equiv m \pmod{N} \\ 0, & \text{aksi hâlde} \end{cases}$$

İspat. $k \neq m$ için toplam, oranı $r = e^{j2\pi(k-m)/N} \neq 1$ olan bir geometrik seridir:

$$\sum_{n=0}^{N-1} r^n = \frac{1 - r^N}{1 - r} = \frac{1 - e^{j2\pi(k-m)}}{1 - r} = 0,$$

çünkü $e^{j2\pi(k-m)} = 1$. $k = m$ ise her terim 1'dir ve toplam N olur. ■

Bu yüzden DFT bir *izdüşümdür*: $X[k]$, işaretin k 'ıncı temel fazör üzerindeki bileşenidir; diğer frekanslar toplama hiç katkı vermez.

```
1 import numpy as np
2 N = 64
3 n = np.arange(N)
4 for k, m in [(5, 5), (5, 9), (5, 69)]: # 69 = 5 + 64
5     ic = np.sum(np.exp(2j*np.pi*k*n/N)*np.exp(-2j*np.pi*m*n/N))
6     print(f"k={k}>2}, m={m}>2} -> {ic.real:8.4f}{ic.imag:+.4f}j")
7 # k= 5, m= 5 -> 64.0000+0.0000j
8 # k= 5, m= 9 -> 0.0000-0.0000j
9 # k= 5, m=69 -> 64.0000+0.0000j <- mod N esitligi
```

39. Fourier Zinciri: Seriden DFT'ye

Dönüşüm	Zaman	Frekans
Fourier serisi (FS)	sürekli, periyodik	ayrık, sonsuz
Fourier dönüşümü (FT)	sürekli, periyodik değil	sürekli, sonsuz
DTFT	ayrık, periyodik değil	sürekli, periyodik (2π)
DFT	ayrık, periyodik (N)	ayrık, periyodik (N)

Zincirin Kuralı

Bir alanda *ayrıklaştırmak*, diğer alanda *periyodikleştirir*. Örnekleme (zamanda ayrıklaştırma) spektrumu f_s periyoduyla tekrarlar — aliasing'in kaynağı budur. Sonlu uzunlukta kesmek (zamanda pencereleme) ise frekansta ayrıklaştırır: DFT'nin bin'leri buradan gelir.

39.1 DTFT'den DFT'ye: Kodun Türetimi

Ayrık zamanlı Fourier dönüşümü

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n}$$

sürekli; bilgisayarda saklanamaz. İki adım atarız: (1) $x[n]$ 'i N örnekle sınırla, (2) ω 'yı N eşit noktada örnekle: $\omega_k = 2\pi k/N$. Sonuç doğrudan DFT tanımıdır:

$$X[k] = \sum_{n=0}^{N-1} x[n]e^{-j2\pi kn/N}.$$

Yani `np.fft.fft`, DTFT'nin N noktada örneklenmiş hâlidir. Sıfır doldurma, aynı DTFT'yi *daha sık* örneklemekten başka bir şey değildir — bu yüzden yeni bilgi katmaz.

39.2 FFT'nin Böl-ve-Fethet İspatı

N çift olsun. Toplamı çift ve tek indislere ayıralım:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r]e^{-j2\pi k(2r)/N} + \sum_{r=0}^{N/2-1} x[2r+1]e^{-j2\pi k(2r+1)/N}.$$

$e^{-j2\pi k(2r)/N} = e^{-j2\pi kr/(N/2)}$ olduğundan:

$$X[k] = E[k] + e^{-j2\pi k/N} O[k].$$

E ve O , $N/2$ periyotludur; ayrıca $e^{-j2\pi(k+N/2)/N} = -e^{-j2\pi k/N}$ olduğu için

$$X[k + N/2] = E[k] - e^{-j2\pi k/N} O[k].$$

Yani $N/2$ noktalık iki dönüşüm, N noktalık dönüşümün tamamını verir. Karmaşıklık bağıntısı $T(N) = 2T(N/2) + O(N)$, çözümü $O(N \log N)$ 'dir. ■

40. Örnekleme Teoreminin Türetimi

Örneklemeyi, işareti bir *darbe treni* ile çarpmak olarak modelleyelim:

$$x_s(t) = x(t) \sum_n \delta(t - nT_s).$$

Darbe treninin Fourier dönüşümü yine bir darbe trenidir; konvolüsyon teoremi gereği çarpma frekansta konvolüsyona dönüşür:

$$X_s(f) = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} X(f - kf_s).$$

Sonuç

Örneklenmiş işaretin spektrumu, orijinal spektrumun f_s aralıklarla *tekrarlanmış* hâlidir. Kopyalar üst üste binmezse ($f_s > 2B$) alçak geçiren bir filtreye orijinali tam olarak geri alırız. Binerse (aliasing) bilgi geri döndürülemez biçimde karışır.

Geri kazanım formülü, ideal alçak geçirenin dürtü yanıtı sinc olduğu için:

$$x(t) = \sum_{n=-\infty}^{\infty} x[n] \text{sinc}\left(\frac{t - nT_s}{T_s}\right).$$

```

1 import numpy as np
2
3 def sinc_yeniden_kur(x, fs, t_yeni):
4     """Whittaker-Shannon enterpolasyonu (ogretici; pratikte resample_poly)."""
5     n = np.arange(len(x))
6     T = 1/fs
7     return np.array([np.sum(x*np.sinc((t - n*T)/T)) for t in t_yeni])
8
9 fs = 1000.0
10 n = np.arange(50)
11 x = np.sin(2*np.pi*70*n/fs)
12 t_ince = np.arange(0, 50/fs, 1/(10*fs))
13 x_ince = sinc_yeniden_kur(x, fs, t_ince)
14 print(np.max(np.abs(x_ince[100:-100] - np.sin(2*np.pi*70*t_ince[100:-100]))))
15 # ~1e-3 -> sonlu ornek sayisindan kaynaklanan uc etkisi

```

41. Konvolüsyon, Modülasyon ve Parseval Teoremleri

Konvolüsyon Teoremi

$$y[n] = \sum_k h[k]x[n - k] \iff Y(f) = H(f)X(f).$$

İspat (DTFT ile).

$$Y(e^{j\omega}) = \sum_n \left(\sum_k h[k]x[n - k] \right) e^{-j\omega n} = \sum_k h[k] e^{-j\omega k} \sum_m x[m] e^{-j\omega m} = H \cdot X,$$

burada $m = n - k$ değişken değiştirmesi yapıldı. ■

Modülasyon (Frekans Kaydırma) Teoremi

$$x[n] e^{j\omega_0 n} \iff X(e^{j(\omega - \omega_0)})$$

Zamanda karmaşık üstelle çarpmak, frekansta *kaydırmaktır*. Mikserin matematiği tam olarak budur.

Gerçek bir kosinüsle çarparsak iki kopya oluşur:

$$x[n] \cos(\omega_0 n) \iff \frac{1}{2} X(e^{j(\omega-\omega_0)}) + \frac{1}{2} X(e^{j(\omega+\omega_0)}),$$

Bu, gerçek mikserlerde *ayna frekansı* (image) probleminin kaynağıdır; IQ mikser (karmaşık çarpım) tek kopya ürettiği için bu sorunu yaşamaz.

Parseval Teoremi

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |X[k]|^2$$

Enerji, hangi alanda ölçülürse ölçülsün aynıdır. Spektral hesaplarını doğrulamanın en pratik yolu budur.

```
1 x = np.random.default_rng(0).normal(size=1024)
2 X = np.fft.fft(x)
3 print(np.sum(np.abs(x)**2), np.sum(np.abs(X)**2)/len(x))
4 # 1030.87... 1030.87... -> esit
```

42. Analitik İşaret ve Anlık Frekans

Anlık Frekans

Analitik işaret $z(t) = A(t)e^{j\varphi(t)}$ için

$$f_{\text{anlık}}(t) = \frac{1}{2\pi} \frac{d\varphi}{dt}.$$

Ayrık zamanda türev, fark ile yaklaşıklanır: $\varphi[n] - \varphi[n-1]$.

42.1 Polar Diskriminatörün Türetimi

$z[n] = A[n]e^{j\varphi[n]}$ olsun. Ardışık örneklerin çarpımı:

$$z[n] z^*[n-1] = A[n]A[n-1] e^{j(\varphi[n]-\varphi[n-1])}.$$

Genlikler pozitif gerçek olduğundan açıyı etkilemez; dolayısıyla

$$\arg(z[n] z^*[n-1]) = \varphi[n] - \varphi[n-1] = \Delta\varphi[n],$$

ve FM için $\Delta\varphi[n] = 2\pi \Delta f m[n]/f_s$ olduğundan çıkış doğrudan mesajla orantılıdır. ■

Bilgi

Bu türetim iki güzel özelliği açıklar: (1) sonuç *genlikten bağımsızdır* — AM gürültüsü FM’i etkilemez, FM’in ünlü “yakalama etkisi” (capture effect) bundandır; (2) `np.angle` sonucu doğal olarak $[-\pi, \pi]$ ’de olduğu için faz sarması sorunu kendiliğinden çözülür, ayrıca `unwrap` gerekmez.

42.2 Hilbert Dönüşümü ile Analitik İşaret

Gerçek $x[n]$ 'in spektrumu simetriktir. Negatif frekansları silmek için frekans alanında bir maske uyguluyoruz:

$$Z[k] = \begin{cases} X[0], & k = 0 \\ 2X[k], & 1 \leq k < N/2 \\ X[N/2], & k = N/2 \\ 0, & k > N/2 \end{cases}$$

Bu tam olarak `scipy.signal.hilbert`'in yaptığıdır.

```

1 def analitik(x):
2     """scipy.signal.hilbert'in acik hali."""
3     N = len(x)
4     X = np.fft.fft(x)
5     h = np.zeros(N)
6     h[0] = 1
7     if N % 2 == 0:
8         h[N//2] = 1
9         h[1:N//2] = 2
10    else:
11        h[1:(N + 1)//2] = 2
12    return np.fft.ifft(X*h)
13
14 x = np.cos(2*np.pi*50*np.arange(1000)/1000)
15 print(np.allclose(analitik(x), sig.hilbert(x)))    # True

```

Neden İki Kat?

Negatif frekansları atınca enerjinin yarısını kaybederiz; pozitifleri iki katına çıkarmak, analitik işaretin gerçek kısmının orijinal işarete *tam olarak* eşit kalmasını sağlar: $\Re\{z\} = x$.

KISIM 10

Referans: NumPy/SciPy Sözlüğü

43. Fonksiyon Sözlüğü

43.1 Spektral Analiz

Fonksiyon	İş
<code>np.fft.fft / ifft</code>	Karmaşık FFT ve tersi.
<code>np.fft.rfft / irfft</code>	Gerçek girdi için yarı boyutlu FFT ($2\times$ hızlı).
<code>np.fft.fftfreq / rfftfreq</code>	Frekans eksenini üretir.
<code>np.fft.fftshift</code>	0 Hz'i ortaya alır (IQ spektrumları).
<code>scipy.fft.next_fast_len</code>	FFT için hızlı uzunluk (2/3/5 çarpanlı).
<code>sig.get_window</code>	Hann, Hamming, Kaiser, flattop...
<code>sig.periodogram</code>	Tek FFT'lik PSD kestirimi.
<code>sig.welch</code>	Ortalamalı PSD (düşük varyans).
<code>sig.spectrogram</code>	Zaman-frekans matrisi.
<code>sig.csd / coherence</code>	Çapraz spektrum, uyum.
<code>sig.find_peaks</code>	Spektrumdaki tepe noktalarını bulur.

43.2 Filtre Tasarımı ve Uygulama

Fonksiyon	İş
<code>sig.firwin / firwin2</code>	Pencere yöntemiyle FIR tasarımı.
<code>sig.remez</code>	Parks–McClellan eşdalgalı FIR.
<code>sig.kaiserord</code>	Gerekli katsayı sayısı ve β kestirimi.
<code>sig.butter / cheby1 / ellip / bessell</code>	IIR prototipleri (output='sos' kullan).
<code>sig.iirnotch / iirpeak</code>	Dar bant çentik / tepe filtresi.
<code>sig.freqz / freqz_zpk</code>	Frekans yanıtı.
<code>sig.group_delay</code>	Grup gecikmesi.
<code>sig.lfilter / lfilter_zi</code>	Nedensel filtreleme, durum taşıma.
<code>sig.sosfilt / sosfiltfilt</code>	Kararlı IIR uygulaması / sıfır faz.
<code>sig.filtfilt</code>	Sıfır fazlı (ileri-geri) filtreleme.
<code>sig.convolve / fftconvolve / oaconvolve</code>	Konvolüsyon çeşitleri.
<code>sig.medfilt</code>	Medyan filtre (dürtü gürültüsü).
<code>sig.detrend</code>	Doğrusal eğilim/DC giderme.

43.3 Hız Dönüşümü ve Modülasyon

Fonksiyon	İş
<code>sig.resample_poly</code>	Polifaz L/M hız dönüşümü (tercih edilen).
<code>sig.decimate</code>	Anti-alias + desimasyon.
<code>sig.upfirdn</code>	Yukarı örnekle–filtrele–indir.
<code>sig.hilbert</code>	Analitik işaret (IQ üretimi).
<code>sig.chirp / sweep_poly</code>	Süpürme işareti üretimi.
<code>sig.square / sawtooth</code>	Kare ve testere dalgası.
<code>sig.max_len_seq</code>	MLS (PRBS) dizisi — dürtü yanıtı ölçümü.
<code>sig.correlate / correlation_lags</code>	Çapraz korelasyon, gecikme kestirimi.
<code>scipy.special.jv</code>	Bessel fonksiyonu (FM spektrumu).
<code>scipy.special.erfc</code>	BER formülleri.

44. Hızlı Tarif Kitabı

```

1 import numpy as np
2 from scipy import signal as sig
3
4 # 1) Ses dosyasından spektrum
5 import soundfile as sf
6 x, fs = sf.read('ses.wav', dtype='float32')
7 f, P = sig.welch(x, fs, nperseg=4096)
8

```

```

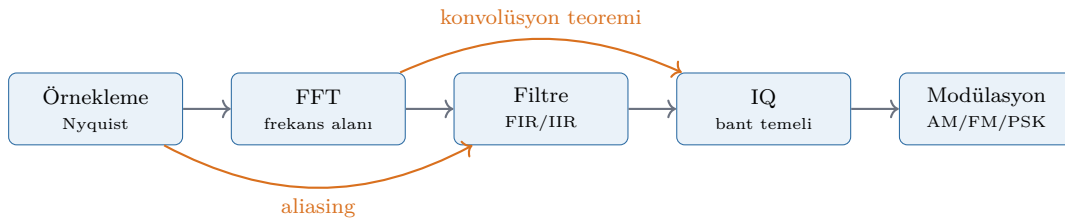
9  # 2) 50 Hz sebekke ugultusunu at
10 b, a = sig.iirnotch(50.0, Q=30, fs=fs)
11 temiz = sig.filtfilt(b, a, x)
12
13 # 3) 300-3400 Hz telefon bandi
14 sos = sig.butter(6, [300, 3400], 'bandpass', fs=fs, output='sos')
15 telefon = sig.sosfiltfilt(sos, x)
16
17 # 4) 48 kHz -> 8 kHz
18 x8 = sig.resample_poly(x, 1, 6)
19
20 # 5) Gercek isaretten IQ (analitik isaret)
21 z = sig.hilbert(x)
22
23 # 6) IQ'yu 100 kHz kaydir
24 n = np.arange(len(z))
25 z2 = z*np.exp(2j*np.pi*100e3*n/fs)
26
27 # 7) FM demodulasyon
28 ses = np.angle(z2[1:]*np.conj(z2[:-1]))
29
30 # 8) Iki isaret arasindaki gecikmeyi bul
31 r = sig.correlate(x, y, mode='full')
32 gecikme = sig.correlation_lags(len(x), len(y), mode='full')[np.argmax(np.abs(r))]
33
34 # 9) Dogru genlikli spektrum tepesi
35 w = sig.get_window('flattop', len(x))
36 X = np.fft.rfft(x*w); genlik = 2*np.abs(X)/np.sum(w)
37
38 # 10) SNR olcumu (ton + gurultu)
39 k = np.argmax(genlik)
40 sinyal_gucu = genlik[k]**2/2
41 gurultu_gucu = np.mean(x**2) - sinyal_gucu
42 print(10*np.log10(sinyal_gucu/gurultu_gucu), 'dB')

```

45. Sık Yapılan Hatalar

Hata	Doğrusu
Filtresiz <code>x[:,M]</code>	Önce anti-alias filtrele; <code>resample_poly</code> kullan.
FFT genliğini N'e bölmek	Tek yanlı genlik $= 2 X /N$ (pencere varsa $\sum w$).
Karmaşık IQ'ya <code>rfft</code>	Karmaşık için <code>fft</code> + <code>fftshift</code> .
Yüksek dereceli IIR'da <code>ba</code>	Her zaman <code>output='sos'</code> .
Bloklar arası <code>zi</code> taşımamak	Her blokta durumu geri yaz; yoksa tık sesi.
NCO fazını sıfırlamak	Fazı biriktir, 2π ile sar.
<code>filtfilt</code>'i canlı akışta	Nedensel değil; yalnız kayıtlı veride.
<code>int16 WAV</code>'ı normalize etmemek	<code>/32768</code> ile ± 1 'e getir.
Örnek başına Python döngüsü	Vektörleştir; kaçınılmazsa <code>numba</code> .
<code>np.angle</code> + <code>unwrap</code> ile FM	<code>np.angle(z[1:]*conj(z[:-1]))</code> daha hızlı ve sağlam.
De-emphasis'i atlamak	FM sesi tiz ve cızırtılı olur.
Pencere kazancını unutmak	Genlik için $\sum w$, güç için $\sum w^2$.

46. Genel Bakış: Zincir Nasıl Bağlanıyor?



Kavram	Sorduğu soru	Python karşılığı
Örnekleme	Kaç hızda ölçmeliyim?	<code>np.arange</code> , <code>fs</code>
DFT/FFT	İçinde hangi frekanslar var?	<code>np.fft.rfft</code>
Filtre	Neyi geçireyim, neyi atayım?	<code>firwin</code> , <code>sosfilt</code>
IQ	Genlik ve faz ayrı ayrı ne yapıyor?	<code>hilbert</code> , <code>np.angle</code>
Desimasyon	Hızı nasıl düşürürüm?	<code>resample_poly</code>
Modülasyon	Bilgiyi taşıyıcıya nasıl bindiririm?	<code>np.exp(1j*faz)</code>
Senkronizasyon	Faz ve saat nasıl kilitlenir?	PLL, Costas, Gardner

İpucu

Öğrenme yöntemi. (1) Her kavramı önce *üret*: bilinen bir işaret sentezle. (2) Sonra *ölç*: FFT'sini al, beklediğin yerde mi bak. (3) Sonra *boz*: gürültü, frekans ofseti, zamanlama hatası ekle ve algoritmanın nerede çöktüğünü gör. (4) En son gerçek veriye geç. Bu sıra izlendiğinde hata ayıklamak kolaydır; tersi sırada ise “çalışmıyor” der ve nedenini asla bulamazsın.